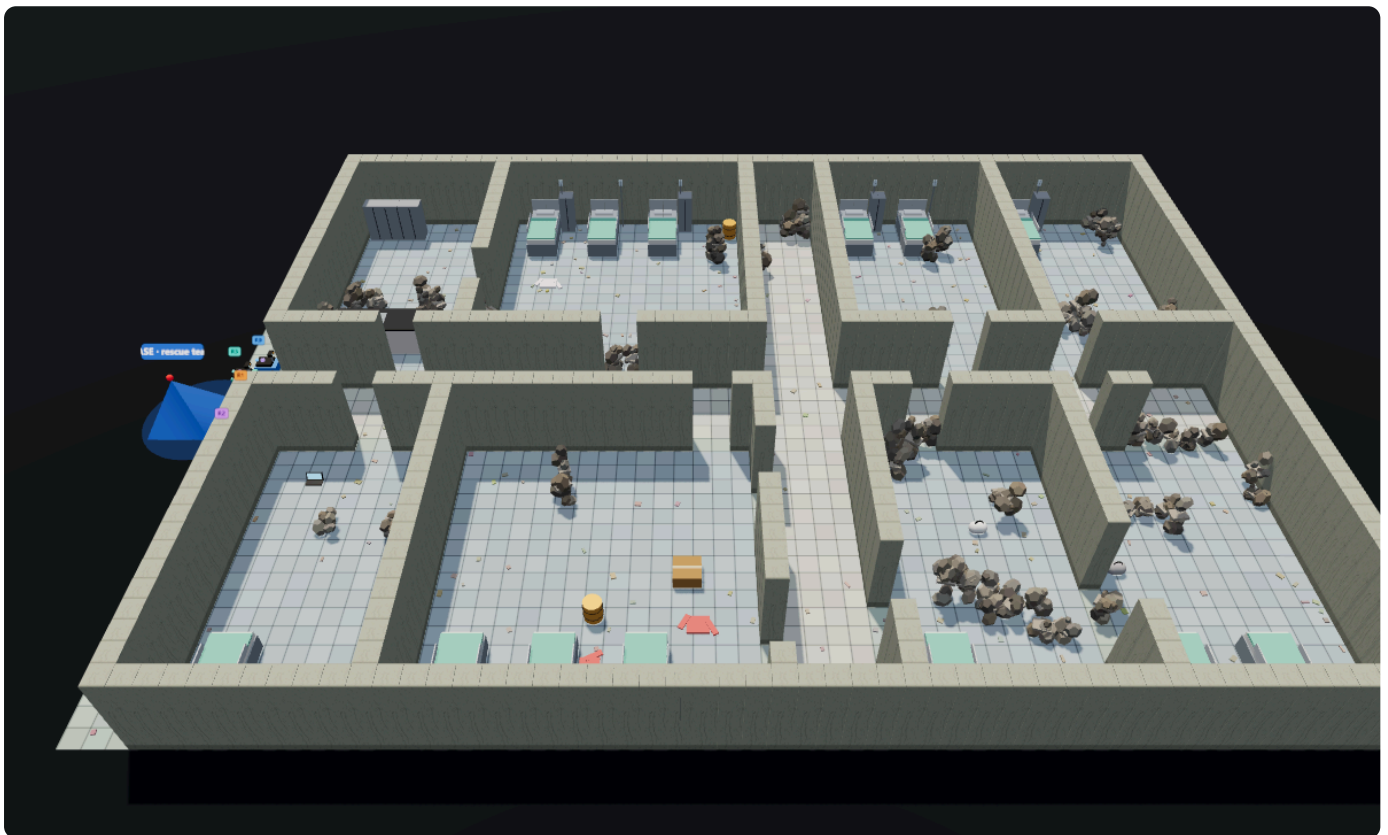


RescueThermal AI

How a team of robots searches an unknown, damaged building for people, explained from first principles, down to the code



What this guide covers. The simulated world, the sensors (LiDAR, colour camera, thermal camera), the maps the robots build, the machine-learning detector, how every decision is made (coverage patterns, team strategies, route planners), how robots react to obstacles, dead ends, teammates and flat batteries, how reliable the whole thing is, every control on the dashboard, the software architecture, and the questions an interviewer is likely to ask.

Written for readers new to robotics. Every number in this guide comes from the project's own code (`rescue/`) and from measurements made with it (`scripts/`, `results/`). Where the simulation simplifies reality, the guide says so.

Contents

1 · The project in one picture

What the robots do · the one-second loop · the big architecture

2 · Robotics basics you need

Grids, cells, coordinates · maps · the robot's body · time

3 · The world: generating buildings

Seven building types · collapse · furniture · rubble · victims · connectivity checks · seeds

4 · Sensors

LiDAR · the RGB-D camera · the thermal camera · what each cannot do

5 · Heat: how temperatures are assigned

Ambient · skin and clothing · rubble blocks heat · warm look-alikes

6 · Maps: from measurements to knowledge

Evidence grids · map vs searched · frontiers · sharing maps

7 · Finding people: the machine-learning detector

Training data · the network · heatmap to position · the belief filter · team verdicts

8 · Deciding where to go

Goals · the decision hierarchy · coverage patterns · team strategies · the auction

9 · Route planning

The grid as a graph · Dijkstra · A* · RRT* · ant colony · comparison

10 · Moving and reacting

Kinematics · teammates · DWA · obstacles · probing · dead ends · revisits

11 · Energy and batteries

The power model · energy-aware tasks · recharging

12 · Every control on the dashboard

What each setting does and how the robots change

13 · Software architecture

Modules · data flow · the server and the 3-D page · tests

14 · How good is it? Measuring

Metrics · reliability study · algorithm checks

15 · Honest limitations

What is simplified and what a real robot would need

16 · Interview questions and answers

64 questions with answers grounded in this code

17 · Glossary and formula sheet

Every term and equation on two pages

HOW TO READ THIS GUIDE

Chapters 1–2 give you the vocabulary. Chapters 3–7 explain what the robots can know about the world and how. Chapters 8–11 explain how they decide and act, which is the heart of the project. Chapters 12–15 connect it to the dashboard, the code and its limits, and chapter 16 is for interview practice. Coloured boxes appear throughout:

- **In simple words:** the idea without jargon.
- **Where in the code:** the file and function that implement it.
- **Interview tip:** how to say it well.
- **Limitation:** where the simulation is simpler than reality.

1 The project in one picture

A building has partly collapsed. Nobody knows its layout any more, or how many people are inside. A small team of ground robots enters through one door. They have to map the building, search every reachable corner with their cameras, recognise the people, and report where they are, without crashing into each other or running out of battery.

RescueThermal AI simulates exactly that mission. It is a complete, working system:

- a **world generator** that builds a new damaged building every time (office, apartments, hospital, school, parking garage, warehouse, or an open test ground), with furniture, rubble, hidden victims and objects that look like people;
- **simulated sensors**: a spinning laser (LiDAR), a colour camera with depth, and a thermal camera;
- **mapping**: each robot builds its own map from noisy measurements, and the team shares maps by radio;
- a **neural network** that looks at the camera images and says where a person might be, plus a statistical filter that decides when a sighting is certain enough to report;
- **decision making** on three levels: a *coverage pattern* (how to sweep the space), a *team strategy* (which robot takes which task) and a *route planner* (how to drive there);
- **motion** with realistic timing, energy use, collision avoidance, dead-end recovery and battery management;
- a **3-D dashboard** in the browser where you configure missions and watch every decision, and scripts that measure how well it all works.

IN SIMPLE WORDS

Think of four people with torches searching a dark, wrecked house. Each person sketches a floor plan as they walk. They phone each other to share sketches and to split the work ("you take the left wing"). When one of them sees something that might be a person, they walk closer to check before shouting "found someone!". That is what the robots do, with a laser instead of a sketch pad and a neural network instead of eyes.

1.1 The one-second loop

The simulation runs in steps of one second. In every step each robot goes through the same five phases. This loop, **sense, detect, communicate, decide, act**, is the heartbeat of the whole project, and nearly every later chapter explains one of its boxes.

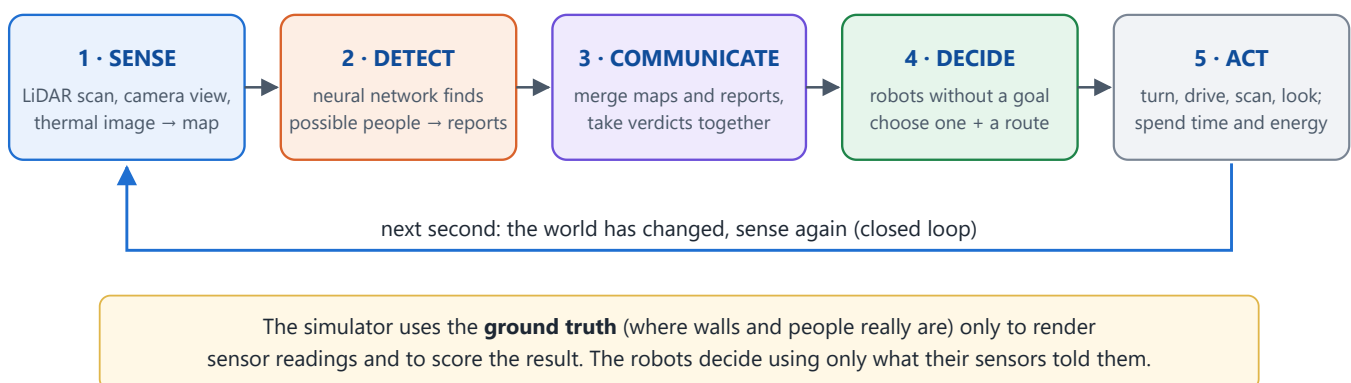


Figure 1.1 The five phases every robot goes through each simulated second (`Simulator.step` in `rescue/simulator.py`).

Two design rules make the simulation honest. **First**, a robot only uses what its own sensors (or its teammates, over the radio) have told it. It does not know where walls are until its laser hits them, and it does not know where people are until a camera sees them. **Second**, the ground truth is used for exactly two things: to generate what a sensor would measure, and to score the mission afterwards (was that "person" really a person?).

1.2 The whole architecture

The next diagram connects every part of the system. Read it from top to bottom. The world produces measurements, measurements become maps and sightings, and three decision layers turn those into motion. Motion changes what the sensors see next.

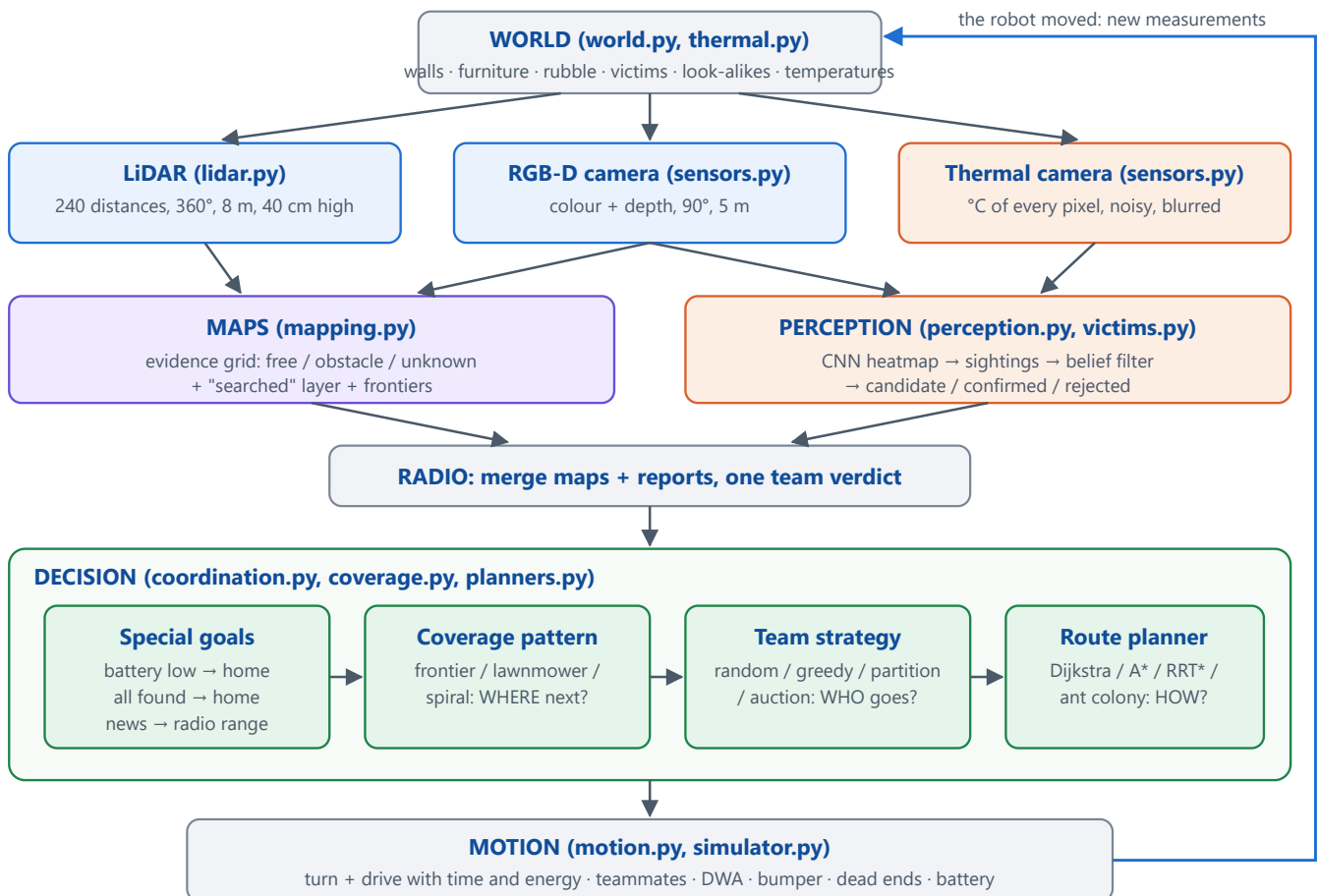


Figure 1.2 The architecture. Each box names the file that implements it. The blue arrow closes the loop: every move changes what the sensors see next.

INTERVIEW TIP: SEPARATE THE QUESTIONS

A strong answer never says "Dijkstra makes the robot intelligent". Each layer answers a different question: *coverage*: which unsearched area next? *strategy*: which robot takes it? *planner*: which route gets there? *motion*: how does the robot physically follow it? *perception*: is that a person? Being able to say which layer does what, and why they are separate, is the single most important thing to take from this guide.

1.3 What a mission looks like

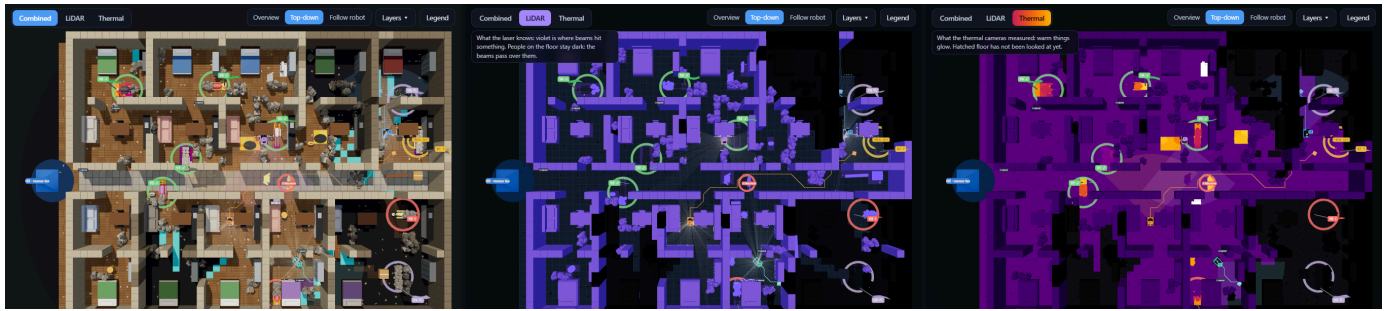


Figure 1.3 One mission, three pictures of it on the dashboard. **Left**, the building as it is, with the robots' findings (green rings are confirmed victims, red X are false alarms). **Middle**, what the lasers mapped. People lying on the floor stay dark because the beams pass over them. **Right**, what the thermal cameras measured. Warm things glow, and hatched floor has not been looked at yet.

A typical mission with 4 robots in the default 28 m × 20 m building takes a few minutes of simulated time. By the end, the whole reachable floor has been searched by a camera, most victims are confirmed, and every robot is back at the base (the blue tent at the entrance, where rescuers wait). The dashboard shows each robot's current goal, its reason in plain words, its planned route and what its planner looked at.

Fact	Value (default)	Where it is set
Floor size	56 × 40 cells of 0.5 m = 28 m × 20 m	<code>RescueConfig.width/height/cell</code>
Simulation step	1 second; a mission ends at 900 s or when every robot is done	<code>max_steps</code>
Robots	4 (1–8 on the dashboard), start at the entrance	<code>n_robots</code>
Victims	10 (hidden number, 4–12, when "unknown" is chosen)	<code>n_victims</code>
Things that look like people	12 look-alikes (jackets, bags...) + 6 warm objects (dogs, heaters...)	<code>n_decoys, n_warm_objects</code>
Driving speed	0.5 m/s, turning 180°/s	<code>speed, turn_rate</code>

2 Robotics basics you need

Five ideas appear on almost every page of this guide: grids, neighbours, coordinates, maps and the robot's body. This chapter explains each one with the exact numbers the project uses.

2.1 The world as a grid of cells

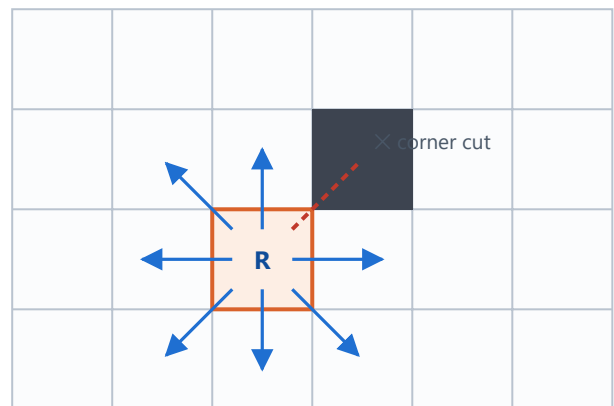
A computer cannot reason about "the floor" as a continuous surface. So the floor is cut into small squares called **cells**, like the squares of a chessboard. Every cell is **0.5 m × 0.5 m**. The default building is 56 cells wide and 40 cells tall (28 m × 20 m). Each cell is either *blocked* (a wall, furniture, rubble, a person lying on the floor) or *free* (a robot can stand there).

Converting between metres and cells is simple arithmetic:

$$\text{cell} = \text{floor}(\text{metres} / 0.5) \quad \text{centre} = (\text{cell} + \tfrac{1}{2}) \times 0.5 \text{ m}$$

So a point at (3.3 m, 7.9 m) lies in cell (6, 15), whose centre is at (3.25 m, 7.75 m). **x grows to the right, y grows downwards** (the convention of images and screens), and angles are measured from the +x axis, so 90° points down the map.

A robot occupies exactly one cell at a time. It knows which cell it is in (see the limitation box below), and it moves from a cell to one of its neighbours.



straight step = 1 cell (0.5 m) · diagonal = $\sqrt{2} \approx 1.414$ cells (0.71 m)

Figure 2.1 The 8 moves from a cell. A diagonal step is only allowed if *both* side cells are free, so the robot never squeezes past the corner of a wall.

2.2 Neighbours and step costs

From any cell a robot can move to its **8 neighbours**: 4 straight (right, left, down, up) and 4 diagonal. A straight step costs **1** (0.5 m of driving) and a diagonal step costs **1.4142** ($\sqrt{2}$, 0.71 m). One rule matters a lot: **no corner cutting**. A diagonal move is only allowed when both of the cells it passes between are free. Without that rule a robot could slip "through" the corner where two walls meet, which a real robot of any width cannot do. This one rule is applied identically by every planner in the project, so their routes can be compared fairly.

WHERE IN THE CODE

`rescue/mapping.py`: `MOVES` lists the 8 moves and their costs; the corner-cutting test `if dx and dy and not (free[y, nx] and free[ny, x])` appears in `dijkstra`, and the same line is in every planner in `rescue/planners.py`.

2.3 A 2.5-D world

The 3-D view looks fully three-dimensional, but the simulation stores the world as a **heightfield**: for every small square of floor, how tall is the thing standing on it? This is called **2.5-D**: height is known, but nothing can float above anything else (no tables with empty space underneath). Two grids describe the same world:

Grid	Cell size	Used for	What each cell stores
------	-----------	----------	-----------------------

Navigation grid	0.5 m (56 × 40)	maps, planning, robot positions	blocked yes/no, tallest height in the cell
Fine grid	0.25 m (112 × 80)	rendering camera images, LiDAR beams, temperatures	height (m), colour (RGB), object id, temperature (°C), rubble cover (m)

Heights matter because the sensors see at different heights. The LiDAR's laser sweeps a flat plane **0.40 m** above the floor: it sees walls, shelves and cars, but its beams pass *over* a person lying on the floor (about 0.2–0.3 m tall). The camera sits at **0.55 m**, looks 14° down, and anything taller than about 0.60 m blocks its view.

IN SIMPLE WORDS

The robots live on a flat floor, so they only need to plan in 2-D (x, y and the direction they face). The building still has real heights, because heights decide what each sensor can and cannot see. That is why a 3-D-looking world with a 2-D planner is a sound design for ground robots, not a shortcut.

2.4 Maps: what a robot believes

There are two kinds of "map" and it is essential not to mix them up:

- The **true world**: where walls and people really are. Only the simulator knows it.
- Each robot's **own map**: what its sensors have told it. Every cell is **UNKNOWN** (never measured), **FREE** or **OBSTACLE**. At the start everything is unknown except the entrance area. This is called an **occupancy grid**, and chapter 6 explains how measurements fill it in.

A robot plans its routes only through cells its own map calls FREE. If the map is wrong (a laser beam missed a low obstacle), the robot finds out when it drives into it: the **bumper** stops it, the cell is marked as an obstacle, and it plans again.

2.5 The robot's body and time

The robots are modelled as 25 kg tracked vehicles with **differential drive**: two tracks, like a tank. Such a robot can turn on the spot by running its tracks in opposite directions, and drives straight when both run at the same speed. In the simulation a move to a neighbouring cell is therefore *turn first, then drive*, and it takes

$$t = |\Delta\theta| / \omega_{\max} + d / v$$

where $\Delta\theta$ is the heading change, $\omega_{\max} = 180^\circ/\text{s}$ is the turning rate, d is 0.5 m (straight) or 0.71 m (diagonal) and $v = 0.5 \text{ m/s}$ is the driving speed. A straight step takes 1 s, a diagonal 1.41 s, and a 90° turn adds 0.5 s. Every simulated second gives each robot one second of "time credit". A move is started while the credit is positive and its duration is subtracted. So over a whole mission a robot drives exactly at its speed, and sharp turns really cost time.

LIMITATION: THE ROBOTS KNOW WHERE THEY ARE

Real robots must estimate their own position (**localisation**) from wheel odometry, an IMU and matching laser scans to the map. Doing that while building the map is called **SLAM** (simultaneous localisation and mapping). This project gives every robot its exact cell. It does *not* simulate SLAM. That keeps the focus on exploration, coordination, perception and planning. Chapter 15 describes what a real deployment would add.

2.6 Random numbers and seeds

Everything random (the building layout, sensor noise, where the victims lie, the random choices inside RRT* and the ant colony) comes from a **pseudo-random generator started from a seed**. The same seed always gives exactly the

same building and, unless you ask the runs to vary, exactly the same mission, move for move. This is what makes fair comparisons possible: to compare A* with Dijkstra, you run both in the same building with the same noise. The project keeps separate random streams for the building (`seed`), the thermal layer (`seed + 7`), sensor noise, planning and tie-breaks (`run_seed`), so changing one never silently changes another.

3 The world: generating buildings

Every mission needs a building nobody has seen before, so that the robots truly explore. `rescue/world.py` builds one procedurally from a seed in about a tenth of a second: a floor plan, damage, furniture, rubble, victims and look-alikes, and then `rescue/thermal.py` gives every surface a temperature.

3.1 The pipeline

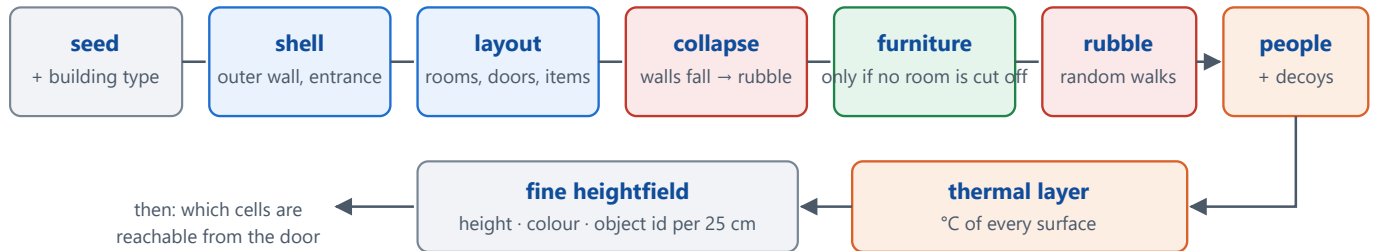


Figure 3.1 How `world.generate(cfg)` builds a building. Every random choice comes from one generator started from `cfg.seed`.

Step 1: the shell

The grid's left three columns ($x = 0-2$) are the **staging area** outside the building, where the robots and the base wait. An outer wall runs around the rest ($x = 3$ to 55 , $y = 0$ to 39) and a **4-cell (2 m) wide entrance** is cut into the left wall at mid-height. The rescuers know this outline (the size of the building), but nothing about its inside, which is exactly what the robots are allowed to know too.

Step 2: the layout of each building type

Each type has its own layout function (`_layout_office`, `_layout_hospital`, ...). It draws interior walls with doors (`hwall` / `vwall` with 2-cell gaps at random positions), paints floor types (room, corridor, special area) and *requests* furniture. Rooms along a corridor are made by `_rooms_along`, which cuts a strip into rooms 8–11 cells wide with one door each onto the corridor.

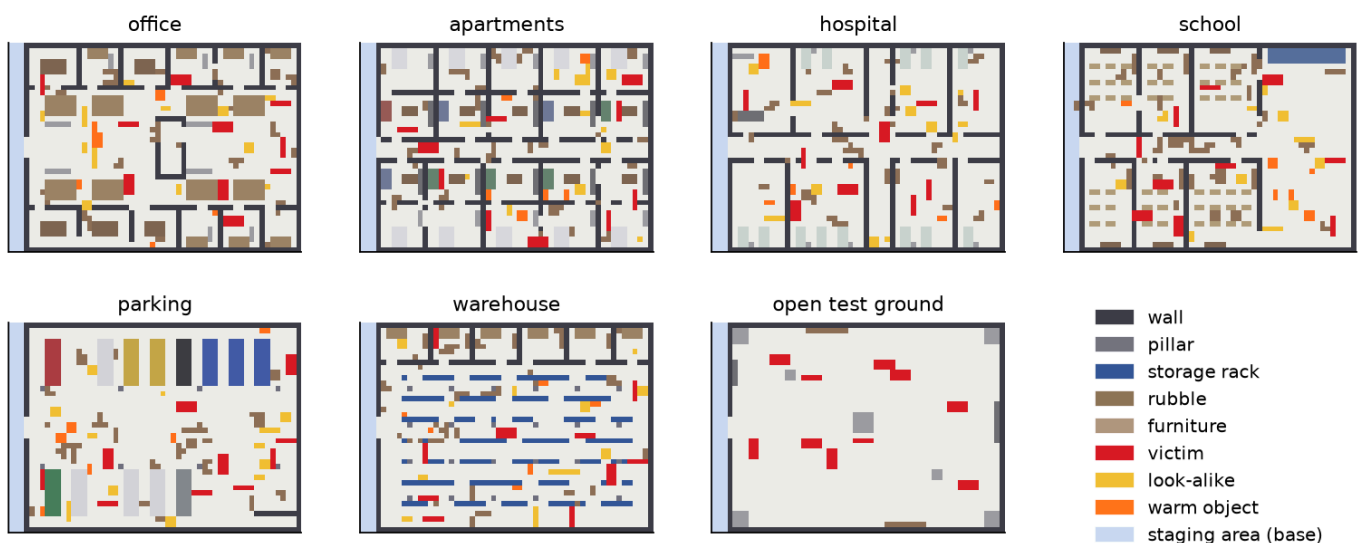


Figure 3.2 One example of each building type (seed 2), as the robots would have to discover it. The legend shows what each colour means. The open test ground has no damage and no look-alikes: it is made for watching the algorithms.

Type	Layout	Furniture that blocks the way	Ambient
Office	rooms along top and bottom walls, open-plan middle, a lift core	desk clusters, meeting tables, cabinets	20 °C
Apartments	central corridor, flats both sides, each with living room and bedroom	sofa, dining table, kitchen counter, bed, wardrobe	21 °C
Hospital	two corridors crossing, wards, reception near the door	hospital beds, bedside cabinets, counter	22 °C
School	classrooms along a corridor, a gym with bleachers	rows of school desks, teacher's table, shelves	19 °C
Parking	two rows of bays, a driving lane, pillars, a stairwell	cars (some with a warm engine)	13 °C
Warehouse	offices along the top, a hall of pillars and storage racks	desks, shelves, pallet racks	15 °C
Open test ground	one hall, fixed obstacles in the middle, corners and along walls	cabinets, shelves, counters	20 °C

Step 3: damage and collapse

The damage level (light / moderate / severe) sets three numbers: the share of interior walls that collapse (10 / 25 / 45 %), the rubble density (3 / 6 / 10 %) and the share of victims partly buried (20 / 40 / 60 %). Collapse picks random interior wall cells as starting points ($share \times wall\ cells / 6$ of them) and removes a run of 2–4 wall cells along the wall from each. Each removed cell becomes rubble 0.4–1.2 m high with probability 35 %. So a collapsed wall can open a new shortcut, or it can fill a doorway with rubble.

Step 4: furniture that never seals off a room

This is the most important safety check in the generator. A randomly placed wardrobe could block the only door of a room, and then nobody could ever search that room. So every furniture request is tested before it is accepted:

```
reach = flood_fill(blocked, from = entrance)      # cells reachable before
trial = blocked + the furniture's cells
after = flood_fill(trial, from = entrance)
accept only if |after| == |reach| - (number of furniture cells)
# i.e. the furniture removed exactly its own cells from the reachable area, nothing more
```

Flood fill is a breadth-first search (BFS) that starts at the entrance and spreads to every free 4-neighbour. If adding the furniture made *more* cells unreachable than the furniture itself covers, it cut something off, and the request is dropped. The same test protects every victim, look-alike and warm object placed later (`_try_place`).

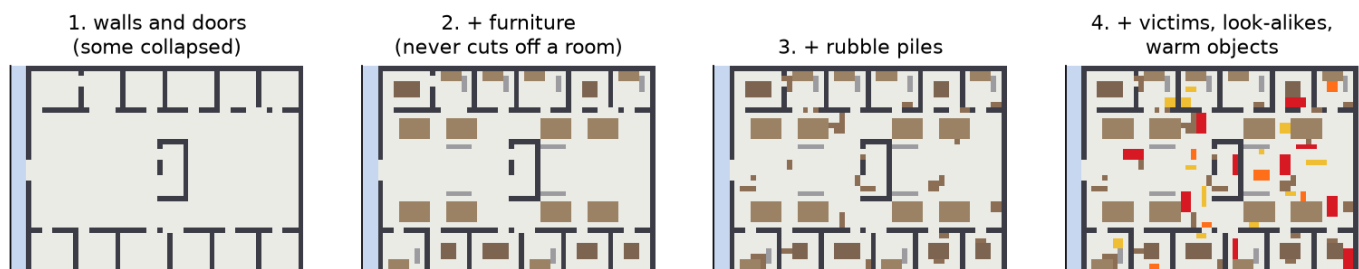


Figure 3.3 The layers of one generated office (seed 3). Collapse has opened some walls. Furniture fills the rooms but never blocks the last way in. Rubble piles grow as random walks. Finally victims (red), look-alikes (yellow) and warm objects (orange) are placed.

Step 5: rubble piles

Piles are grown as short **random walks**: from a random free cell, 2–6 steps in random directions, each visited cell getting rubble 0.3–1.4 m high. The number of piles is $\text{density} \times \text{rubble slider} \times \text{free cells} / 4$. Rubble is never placed on walls or furniture, and never within 4 cells of the entrance (the robots must be able to get in).

Step 6: people, look-alikes and the fine heightfield

A **victim** is a body 7×2 fine cells (1.75 m $\times$ 0.5 m) lying horizontally or vertically. It has a head, a torso in a shirt colour and legs in a trouser colour, and heights of about 0.2, 0.28 and 0.18 m. Victims are kept at least 2 m apart, and each placement must pass the same "cuts nothing off" test. Depending on the damage level, part of a body is covered by rubble 0.35–0.6 m thick (2–4 of its 7 segments). The thermal step later may bury some bodies completely (chapter 5).

Look-alikes (called decoys in the code) are things that can fool a colour camera: a jacket (flat, shirt-coloured), a bag, a cardboard box, a wooden beam and an orange barrel. Finally light floor debris (4 % of fine cells, 3–8 cm high) is scattered for visual clutter. Robots drive over it.

WHERE IN THE CODE

`world.generate` runs the pipeline; `LAYOUTS` maps each building name to its layout function; `_flood` is the flood fill; `_try_place` places bodies and objects without cutting anything off; `_place_people_and_decoys` builds bodies segment by segment. The world object (`World`) holds `blocked`, `height`, `color`, `obj`, `temp`, `cover`, the victims and the decoys.

3.2 The open test ground

The test ground (building *plain*) is deliberately **not random**. It is one empty hall with a 2×2 m block in the centre, two crates on the diagonal, 1.5 m cabinets in all four corners, shelves along the top and bottom walls and counters on the side walls, plus a faint painted 2 m grid. It has no collapse, rubble, look-alikes or warm objects. Only the victims change with the seed. That makes it the right place to check that a lawnmower really drives straight lanes, or that A* really finds the same route length as Dijkstra.

INTERVIEW TIP: "HOW DO YOU GUARANTEE EVERY AREA CAN BE SEARCHED?"

"Every object is placed tentatively and accepted only if a flood fill from the entrance shows that the reachable area shrank by exactly the object's own cells. So no placement can seal off a room, and the metric 'share of reachable floor searched' always has a well-defined 100 %." Mention the seed too: the same seed reproduces the same building, which is what makes experiments fair.

4 Sensors

Every robot carries three sensors that complement each other. The **LiDAR** measures geometry all around. The **RGB-D camera** sees colour and distance ahead, including low objects. The **thermal camera** sees warmth. None of them "knows" anything. Each produces numbers that later stages must interpret.

Sensor	Sees	Range / field	Good at	Cannot
LiDAR	distance to surfaces crossing a plane 40 cm up	8 m, 360°, 240 beams (1.5° apart)	fast, accurate (± 2 cm) maps of walls, in the dark	see anything lower than 40 cm (a lying person); tell a person from a box
RGB-D camera	colour + depth of every pixel	5 m, 90° wide, 64 × 48 px, 0.55 m high	low objects on the floor; what things look like	see in the dark without its headlamp; see through dust well; see behind furniture
Thermal camera	surface temperature of every pixel (°C)	same view as the colour camera	warm bodies, in the dark, through dust	see through thick rubble; tell a dog (33 °C) from a person

4.1 LiDAR: a spinning laser

A real LiDAR fires a laser pulse and times the echo: distance = speed of light × round-trip time / 2. In the simulation there is no light to time, so the same result is computed by **ray casting**: for each of the 240 beam directions, walk along the beam and find the first place where the world is taller than the scan plane.

```

for every beam angle a (240 of them):
  walk from the robot in steps of 8 cm:  $p = \text{robot} + t \cdot (\cos a, \sin a)$ 
  stop at the first sample whose fine-grid height > 0.40 m (a surface crossing the scan plane)
  refine the last step to 1 cm (find the surface as precisely as a real laser)
  measured = true distance + Gaussian noise ( $\sigma = 2$  cm)
  with 1 % probability: the beam returns nothing (dropout)
  
```

A beam gives two kinds of information. Every cell it passed through is probably **free**, because light went through it. The cell where it ended probably contains a **surface**. Turning beams into this cell-level evidence is called the **inverse sensor model**. The project counts, for every cell, how many beams passed through it and how many ended in it. A cell is "hit" only if more beams ended in it than passed through it. Two small margins make this robust to 2 cm noise: a beam proves free space only up to 6 cm before its measured end, and its hit is placed 3 cm behind the surface, so a beam that grazes a wall corner doesn't end in the empty cell beyond.

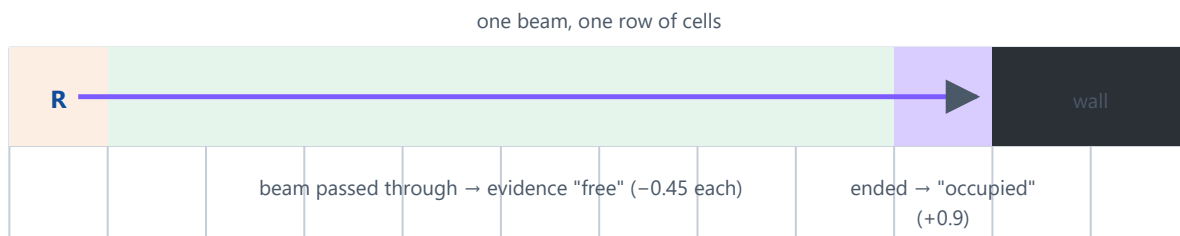


Figure 4.1 The inverse sensor model. Chapter 6 shows how these little amounts of evidence add up over many scans into a reliable map.

one turn of the laser: 240 beams, 8 m, 40 cm above the floor what that scan says: green = beams passed (free), violet = beams ended (hit)

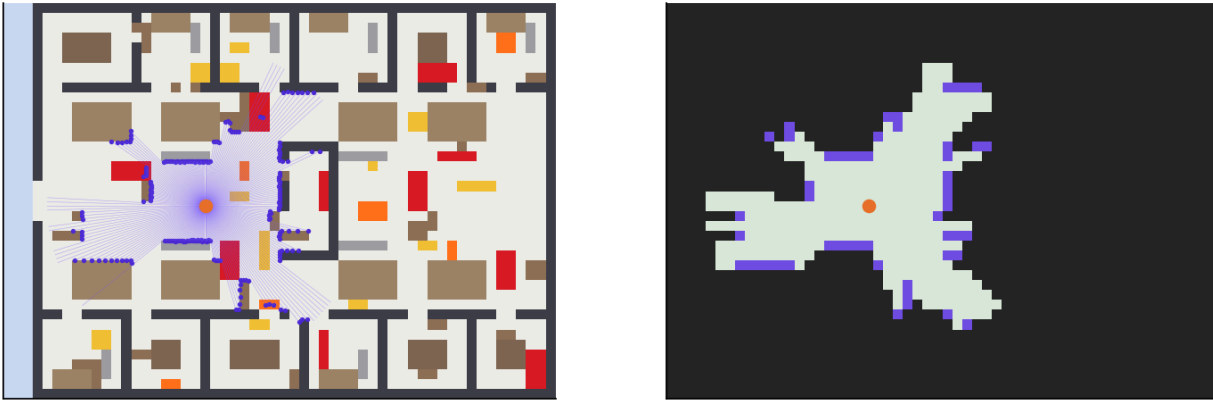


Figure 4.2 Left: one real scan in an office. Violet lines are the 240 beams, dots are where they ended. Right: what that single scan says about each cell. Note the "shadows" behind furniture: the laser cannot see through things, so the robot must move to see behind them.

IN SIMPLE WORDS

The laser is like touching everything around you with 240 very long sticks at waist height, once a second. You learn exactly where walls are, but a person lying on the floor is below your sticks, and a stick cannot tell you whether it touched a box or a human.

4.2 The RGB-D camera

The colour camera is a **pinhole camera** model: 64×48 pixels, 90° field of view, mounted 0.55 m above the floor and tilted 14° down, lit only by the robot's headlamp because the building is dark. Each pixel is rendered by **ray marching**:

1. From the camera's focal length $f = (W/2) / \tan(FOV/2)$, each pixel (u, v) defines a direction; the tilt rotates it down; the robot's heading rotates it into the world.
2. March along the ray in 7 cm steps until the ray's height drops below the height of the fine-grid cell it is over (it hit a surface) or below the floor.
3. The pixel's **depth** is the distance travelled; its **colour** is the surface colour $\times$ shading (top faces bright, side faces 70 %) $\times$ a headlamp fall-off $0.04 + 1.15 / (1 + 0.09 d^2)$; its **3-D point** is where the ray stopped. That point is how a detection is later placed on the map.
4. Sensor imperfections: dust haze grows with distance, exposure varies by $-20\% / +15\%$ per frame, and every pixel gets Gaussian noise.

For the **map** and for measuring what has been **searched**, a cheaper geometric version is used (**visible_cells**): a fan of rays every 1.5° across the 90° field, each stopping at the first cell taller than the camera. Every cell a ray reached counts as "seen by a camera". This defines **coverage**: the share of reachable floor some camera has looked at.

4.3 The thermal camera

A thermal camera measures infrared radiation and converts it to temperature. In the simulation it is mounted next to the colour camera and uses the same rays. Each pixel reports the **temperature of the fine-grid cell its ray hit** (chapter 5 explains where those temperatures come from). Then it is made realistic:

- **dust between camera and object** pulls the reading $6\% \times (1 - e^{-d/4 m})$ towards the ambient temperature;
- **soft optics**: a Gaussian blur of 0.8 pixel (thermal lenses are blurrier than colour ones);

- **sensor noise** of 0.25 °C per pixel, plus a **calibration drift** of about 0.3 °C shared by the whole frame, as in a real uncooled thermal core.

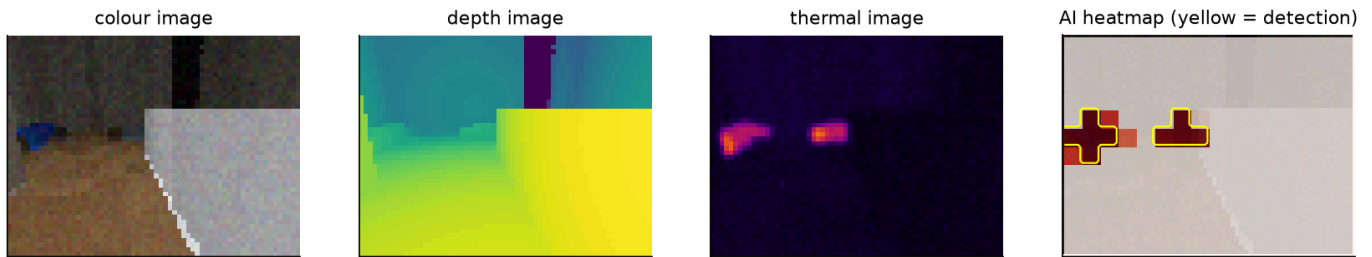


Figure 4.3 What one robot sees in one frame, all rendered by `sensors.render`: the colour image (headlamp only), the depth image (dark = near), the thermal image (two warm body parts glow), and the detector's heatmap on top (chapter 7). Yellow outlines are the blocks above the detection threshold.

LIMITATION: TEMPERATURES ARE SIMULATED

No real heat exists inside a computer. The thermal images are **synthetic measurements from a sensor model**: each surface has an assigned temperature, and the camera model adds the effects a real camera has (blur, noise, drift, haze). Real thermal cameras also depend on *emissivity* (shiny metal reads colder than it is), reflections and humidity, which the model does not include.

WHERE IN THE CODE

`rescue/lidar.py` `scan()`: beams, noise, dropout and the pass/hit counting. `rescue/sensors.py`: `_camera_rays` (pinhole geometry with tilt), `render` (ray marching → colour, depth, 3-D points, thermal image), `visible_cells` (what the camera covers on the map), `line_of_sight` (used for close looks and radio).

5 Heat: how temperatures are assigned

A thermal camera is only as meaningful as the temperatures it looks at. `rescue/thermal.py` gives every 25 cm square of the building a surface temperature in °C, following simple physical rules. The rules are chosen so that the thermal camera faces the same difficulties as a real one: warm things that are not people, and people it cannot see.

5.1 The building itself

Each building type has an **ambient temperature**: office 20 °C, apartments 21, hospital 22, school 19, parking 13, warehouse 15, test ground 20. Collapsed walls let outside air in, so damage cools it: -0.5 °C (light), -1.5 °C (moderate), -3 °C (severe). On top of that:

- **drift**: a smooth random field (Gaussian-blurred noise, $\sigma = 6$ fine cells = 1.5 m) scaled to ± 0.8 °C, for draughts and sunlit patches;
- **noise**: ± 0.15 °C per square;
- **walls and structure** are 0.6 °C cooler (concrete and steel).

5.2 People

Each victim gets a skin temperature drawn between **33 and 36 °C**. Half of the victims have been trapped for hours and have cooled by up to 4 °C (**hypothermia**), so a victim can be as cool as 29 °C. The head segment shows the skin temperature. Torso and legs are covered by clothing, which insulates and shows **4–7 °C cooler** than the skin.

5.3 Rubble blocks the heat

This rule decides which victims can be found at all. Heat leaking through a cover of thickness d falls off exponentially:

$$T_{\text{surface}} = T_{\text{ambient}} + (T_{\text{body}} - T_{\text{ambient}}) \cdot e^{-d / 0.10 \text{ m}}$$

Cover on the body	Thickness d	Share of the body's warmth at the surface	Who can find this person?
none	0	100 %	colour and thermal cameras
thin dust / light debris	6–14 cm	55 % ... 25 % (e.g. 10 cm $\rightarrow e^{-1} = 37$ %)	only the thermal camera (a faint warm patch)
partly buried	35–60 cm on 2–4 of 7 segments	the uncovered segments: 100 %	both, if an uncovered part is in view
thick concrete rubble	40–80 cm over the whole body	2 % ... 0.03 %	nobody : no camera can see through concrete

How many victims are fully covered depends on the damage: light 5 % thin / 0 % thick, moderate 10 % / 10 %, severe 10 % / 20 %. People under thick rubble are **undetectable by design**. The project reports them separately ("victims buried deep") so its results never pretend that a camera can see through concrete. Finding them would need listening devices, search dogs or rescuers digging.

5.4 Warm things that are not people

A colour camera can be fooled by a jacket; a thermal camera by anything warm. Six warm objects per building (chosen to suit the building type) give the thermal detector realistic hard cases:

Warm object	Temperature	Buildings	Why it is hard
space heater	42–60 °C	office, apartments, school, hospital	hot, but caught by the 40 °C rule
laptop left running	34–42 °C	office, school, hospital, warehouse	small, person-warm
hot-water leak (puddle)	28–38 °C	apartments, hospital, office, school	large warm patch; robots can drive through it
dog	31–36 °C	apartments, parking, warehouse, school	same temperature as a person, body-like shape
sun-warmed rubble	28–36 °C	all	warm, irregular, lies where victims lie
car engine that ran recently	40–65 °C (bonnet)	parking (40 % of cars)	big and warm; usually too hot

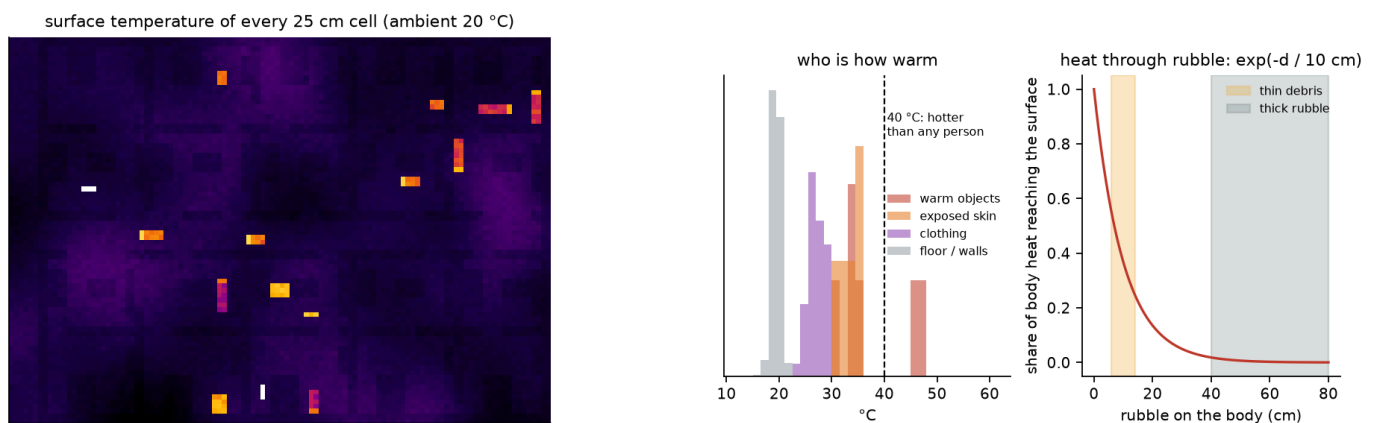


Figure 5.1 Left: the temperature field of an apartment block in the "ironbow" palette used by thermal cameras (black = cold, white = hot). Bodies, a dog and hot water glow; the walls are slightly cooler. Middle: the temperature ranges overlap. Clothing, skin, dogs and warm rubble share 28–36 °C, which is why temperature alone cannot identify a person. Right: how quickly rubble blocks heat.

5.5 The one physics rule the robots use

Nothing on a living person is hotter than about **40 °C**. So if the detector says "person" but the hottest pixel of that detection is above 40 °C, the detection is dropped as a heater or an engine (`thermal_check` , `HUMAN_MAX`). This rule cannot remove a dog or warm rubble, because they sit *inside* the human range. Those are left to the neural network (shape and appearance) and the belief filter (repeated, close looks), chapter 7.

INTERVIEW TIP: "IS THE TEMPERATURE REAL?"

"No. It is a physically-motivated simulated temperature field: ambient per building, skin 33–36 °C with possible hypothermia, clothing 4–7 °C cooler, heat through rubble falling as $e^{-d/10 \text{ cm}}$, and warm look-alikes with realistic ranges. A camera model then adds blur, noise, drift and haze. The point is that the thermal detector has to cope with the same ambiguities a real one does: a dog at 33 °C, and a person under 50 cm of concrete with no signature at all."

WHERE IN THE CODE

`rescue/thermal.py`: `add_thermal` (ambient, drift, walls), `_bury_and_warm_victims` (skin, clothing, covers, heat leak), `_place_warm_objects`, `_warm_car_engines`, `palette` (ironbow colours). It uses its own random stream (`seed + 7`), so adding heat never changed any building layout.

6 Maps: from measurements to knowledge

A single laser beam can be wrong: noise, a dropout, a beam grazing a corner. So a robot never trusts one measurement. It keeps an **evidence grid**, a probabilistic occupancy map in which every measurement adds a little evidence, and it only calls a cell free or blocked once the evidence is clear.

6.1 Log-odds: adding up evidence

For each cell the robot keeps a number L , the **log-odds** that the cell is occupied: $L = \ln(p / (1 - p))$, where p is the probability of "occupied". $L = 0$ means 50/50 (no idea), positive means probably occupied, negative probably free. Log-odds are used because independent pieces of evidence can simply be **added** (the Bayesian update becomes addition). This is the classic occupancy grid of Moravec and Elfes (1985).

Event	Change to L	Why this size
a laser beam passed through the cell	-0.45	weak: a beam could slip past a thin obstacle
a laser beam ended in the cell	+0.9	stronger: a return means something is there
the laser alone	never beyond ± 2.5	so closer, better sensors can always overrule it
the depth camera saw the cell from close by	set to -4 (free) or +4 (blocked)	the camera also sees low objects the laser misses; final
the bumper hit it / the robot stands on it	+4 (proven solid) / -4 (proven free)	physical contact is certain

A cell is **FREE** when $L \leq -0.8$, **OBSTACLE** when $L \geq +0.8$, and **UNKNOWN** in between. Example: two laser beams pass through a cell $\rightarrow L = -0.9 \rightarrow$ FREE. A cell where one beam ended and two passed: $+0.9 - 0.9 = 0 \rightarrow$ still UNKNOWN, and the robot waits for more evidence. Cells the camera or bumper proved solid are flagged, and later laser beams can never free them again. That matters for a jacket on the floor: every laser beam flies over it, but the camera saw it.

IN SIMPLE WORDS

Each cell has a score. Laser beams flying through it nudge the score towards "free", beams stopping in it nudge it towards "blocked". Only when the score passes a threshold does the robot believe it. Its own eyes (the camera) and its own body (the bumper) are trusted more than the laser.

6.2 Two questions, two layers

The map answers "*can I drive there?*". But a search-and-rescue robot must also answer "*has anyone looked there for people?*" Only a camera can answer the second question, because the laser cannot recognise a person. So every robot keeps a second layer, **searched**: the cells a camera (its own or a teammate's) has seen. The laser usually maps much further than the cameras have searched, so the difference is large (grey areas in Figure 6.2).

For exploration the robot uses a combined view (`explore_map`): a cell counts as FREE or OBSTACLE *only if it has also been searched*, and as UNKNOWN otherwise. In effect, "unknown" means "still to be searched for people". That includes obstacles only the laser has seen, because a rubble heap the laser bounced off may have a person lying behind or under it.

6.3 Frontiers: the edge of what has been searched

A **frontier cell** is a searched free cell next to (up, down, left or right) a cell nobody has searched yet. It is the boundary between known and unknown. Driving to a frontier and looking around is guaranteed to reveal something new. This is the idea behind **frontier-based exploration** (Yamauchi, 1997), the most widely used autonomous exploration method.

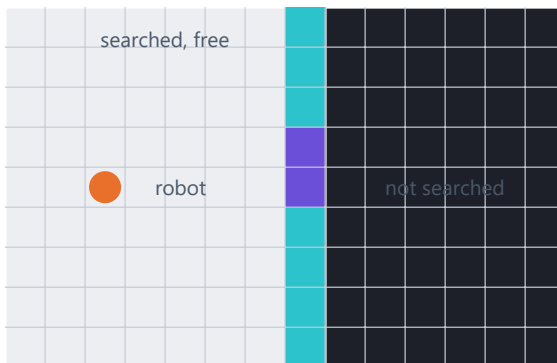


Figure 6.1 Frontier cells (cyan) lie between searched floor and unsearched space. The violet cell is an obstacle, so it is not a frontier. This long frontier would be cut into pieces of about 10 cells, each one a possible goal.

Frontier cells are grouped into **clusters** (connected 8-way, `scipy.ndimage.label`). A long cluster, say a whole wall of an unexplored hall, is cut into **pieces of about 10 cells** along its main direction (found with an SVD, the direction in which the cells spread most). Then several robots can work along one big opening. Each piece becomes one possible goal:

- **goal cell:** the reachable cell of the piece closest to its centre (the exact centre may be unreachable);
- **value:** the number of frontier cells in the piece, a proxy for how much new area it should reveal;
- **distance:** the route length from the robot (from its distance field, chapter 8).

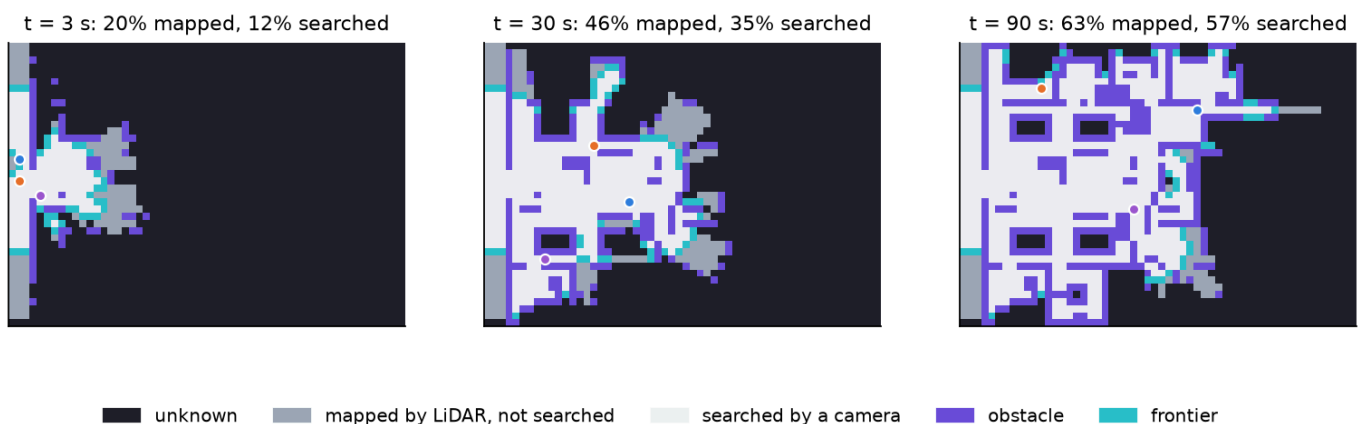


Figure 6.2 The team map of three robots in an office at 3 s, 30 s and 90 s. The LiDAR maps quickly (grey = mapped but not yet searched); the cameras follow (white = searched). Cyan frontiers mark where searching should continue. By 90 s the robots are spreading into the rooms along the top and bottom.

6.4 Sharing maps without double counting

Robots that can talk by radio merge their maps every second. Merging must not simply add the evidence: if robot A told robot B about a wall yesterday, and B tells A today, adding would count the same observation twice and make the map over-confident. So the merge keeps, for every cell, the **strongest single opinion** (the evidence with the largest $|L|$), and any cell someone proved solid stays an obstacle. The searched layers are merged with a logical OR, and so are the "given up" cells (next section).

6.5 When the sensors cannot resolve a cell

Sometimes a robot stands on a frontier and still cannot tell what is in the cell next to it: a table top blocks the laser, a door frame makes the laser call a passable cell an obstacle, the camera cannot see into a corner. Then the robot **probes**: it drives slowly into that cell. Either it gets in (the cell is free; a robot knows the floor it stands on) or the

bumper stops it (an obstacle, now proven). A cell still unresolved after one probe is **given up**: it no longer counts as a frontier, and the team shares that decision. This guarantees the search always ends.

WHERE IN THE CODE

```
rescue/mapping.py: apply_lidar, apply_camera, classify (evidence → FREE/OBSTACLE/UNKNOWN),  
merge_evidence (strongest opinion), explore_map (the searched view), frontiers (frontier clusters).  
rescue/coordination.py: _pieces (cutting frontiers with an SVD) and robot_goals. Probing:  
Simulator._probe.
```

7 Finding people: the machine-learning detector

Recognising a person in a noisy, dark, partly buried scene is the one job in this project that simple rules cannot do well. A temperature threshold would flag every dog and puddle of hot water, and a colour rule would flag every red jacket. So the project trains a small **convolutional neural network (CNN)** on images rendered by the simulator itself, then wraps its output in a statistical filter that only reports a person after repeated, close looks.

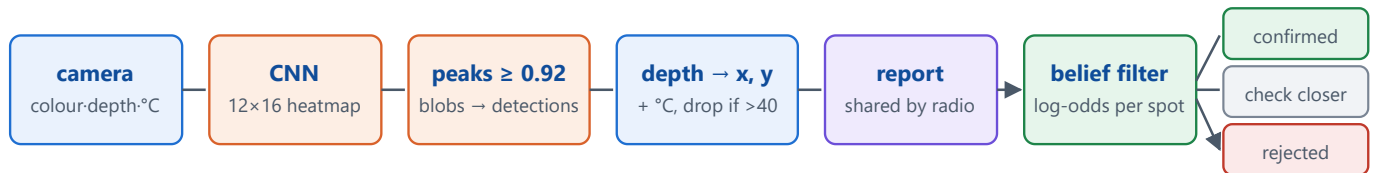


Figure 7.1 From pixels to "victim found". Only the last step says "found"; everything before it produces evidence.

7.1 Three detectors, one network

The same network architecture is trained three times, once per sensor set on the dashboard. The only difference is what it is given as input:

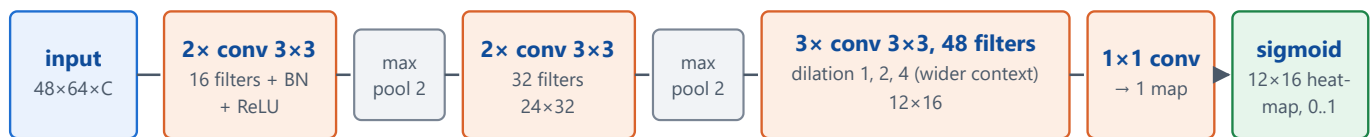
Detector	Input channels	Sees people by...	Typical confusion
colour camera (<i>cnn</i>)	R, G, B, depth (4)	shape and colours of head, shirt, trousers	bags, jackets, beams
thermal camera	depth, temperature (2)	a warm, body-sized shape	dogs, sun-warmed rubble, hot water
fused (colour + thermal)	R, G, B, depth, temperature (5)	both at once	fewest; dogs remain the hardest case

7.2 Training data made by the simulator

There is no dataset of labelled images of collapsed buildings with victims. So the simulator makes one. It generates **200 random disaster zones** (random building type, damage level, 8–14 victims, 10–19 look-alikes, 4–9 warm objects) and renders **100 camera views** in each, 20,000 images in all. The views are deliberately biased towards the interesting cases: about a third aimed at a victim from 0.8–5 m, a third aimed at a look-alike or a warm object (the *hard negatives*), and a third at random.

Labelling is automatic, because the renderer knows which object every pixel shows. The image is cut into 4 × 4-pixel blocks (a 12 × 16 grid), and a block is labelled "person" when at least 15 % of its pixels show a victim that *this sensor could actually see*: exposed body parts for the colour detector, and also parts under thin debris (< 20 cm, heat still leaks through) for the thermal and fused detectors. No network is asked to see through concrete. A separate test set of 40 disaster zones never used in training (4,000 images) measures how well each detector generalises.

7.3 The network (VictimNet)



about 73,000 weights in total · fully convolutional: every output cell says "how likely does this 4×4 block show part of a person?"

Figure 7.2 The detector network (`perception.build_net`). Two poolings shrink 48 × 64 pixels to a 12 × 16 grid; dilated convolutions then let each output cell "see" a larger part of the image without shrinking it further.

Why this design? A **fully convolutional** network outputs a heatmap instead of a single yes/no, so one image can contain several people and each gets a position. **Batch normalisation** keeps training stable. **Dilated** convolutions widen the receptive field cheaply: dilation 4 on a 12 × 16 map lets a cell take in almost the whole body, which is needed to tell a lying person (1.75 m) from a 0.75 m dog. The whole network has only about 73,000 weights, so it runs on a CPU in milliseconds per frame. It is *not* YOLO or a pretrained model: it is trained from scratch on the simulator's images.

Training

- **Loss:** binary cross-entropy on every block (`BCEWithLogitsLoss`). Person blocks are rare (a few % of blocks), so they are up-weighted by $\sqrt{((1 - p)/p)}$, capped at $\sqrt{20}$. Without that, the network could score well by always saying "nobody".
- **Optimiser:** AdamW, learning rate 0.002 with cosine decay, weight decay 10^{-4} , batches of 64, **20 epochs**.
- **Augmentation:** half the batches are mirrored left-right (a person is still a person in a mirror).

Detector (on 4,000 unseen test images)	ROC-AUC (image)	Heatmap average precision	Image recall	Image precision
colour camera	0.969	0.930	93.0 %	71.5 %
thermal camera	0.990	0.980	98.4 %	81.6 %
colour + thermal fused	0.994	0.993	98.3 %	83.0 %

ROC-AUC: the probability that a random image with a person scores higher than a random image without one (1.0 = perfect). Image precision/recall at a 0.5 threshold. These single-image scores are only the start: the belief filter below combines many images, which is what makes the team's reports far more reliable than any single frame.

7.4 From heatmap to a position on the map

Every other second (every second while scanning or taking a close look) each robot runs its detector on its latest frame:

1. Blocks with heat ≥ 0.92 are grouped into connected blobs; each blob is one detection with the confidence of its peak block.
2. Inside the peak block, the **nearest depth reading** gives the distance, and the renderer's 3-D point for that pixel gives the **map position (x, y)**. This is exactly what a real RGB-D camera provides: pixel + depth $\rightarrow$ 3-D point.
3. With a thermal camera, the hottest pixel of the block is attached. If it is above 40 °C, the detection is dropped (a heater or an engine, chapter 5).
4. For every open candidate that is in clear view within 3 m but produced no detection, a **negative report** ("I looked there and saw nobody") is sent.

7.5 The belief filter: why one image is never trusted

Reports within 1.3 m of each other are fused into one **candidate** with an evidence score in log-odds (the same idea as the map in chapter 6):

Event	Evidence change
a new candidate starts at	−0.6 (most first glimpses are wrong)
a detection with confidence p , seen from ≤ 2.5 m ("close look")	$+ 1.0 \times \ln(p / (1 - p))$
the same, from further away	$+ 0.55 \times \ln(p / (1 - p))$
a close look that saw nobody	− 0.9 ($\times 0.55$ from further away)

A candidate is **CONFIRMED** only when **all three** hold: evidence ≥ 9 , seen in ≥ 6 different camera frames, and at least **one close look** (≤ 2.5 m). It is **REJECTED** when its evidence falls to -1.5 . Uncertain candidates become **verify tasks**: a robot drives to a spot 0.7–2.2 m from the candidate with a clear line of sight, looks at it for 2 s, and if that was inconclusive, the next look is taken from a different side (a buried person may be visible from one angle only), up to 4 looks.

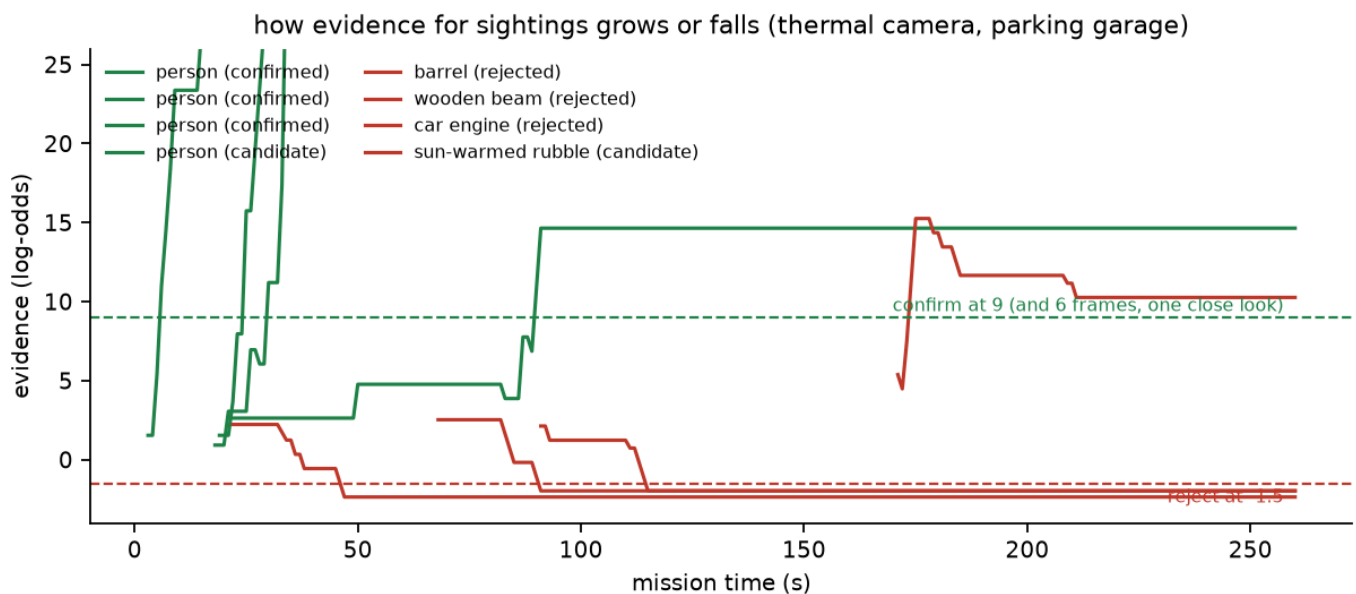


Figure 7.3 Evidence for sightings during one thermal mission. Green lines are real people: repeated close looks push them past 9 quickly. Red lines are other things: a barrel, a beam and a warm engine sink to the rejection line once robots look closely. One sun-warmed rubble pile rose above 9 but was not confirmed, because the other conditions (6 frames, a close look) were not met. That is exactly what they are for.

7.6 One verdict for the whole team, one report per person

Evidence is a plain sum, so the order of reports does not matter. But *when* a verdict is taken does. The project therefore takes verdicts once per second, after the robots have exchanged reports, and every robot applies new reports in the same order (by report id). Robots that hold the same reports then always hold the same verdicts. (Before this rule, in 6 test missions one robot rejected a spot another had confirmed 7 times; now it is 0.) A confirmation is final; a rejection is not: new, stronger evidence can reopen it.

Finally, a lying body is 1.75 m long and its head and legs can look like two warm shapes. Confirmed sightings closer than 1.6 m are treated as the **same person**. If they are further apart, the dashboard labels the second one "☐ counted twice", which is a counting error, not a phantom.

INTERVIEW TIP: "HOW DOES ML FIT INTO THE ROBOT?"

"ML does not drive the robot. It turns images into evidence: a heatmap of where a person might be. Classical robotics does the rest: depth turns a pixel into a map position, a Bayesian filter accumulates evidence over many frames and viewpoints, and the decision layer sends a robot for a closer look when the evidence is uncertain. The navigation stack itself is entirely algorithmic."

WHERE IN THE CODE

`rescue/perception.py`: `build_net`, `make_dataset`, `train_detectors`, `evaluate`, `CNNDetector.detect`.
Training: `python -m rescue train` (models saved in `models/*.pt` with their test scores in `.json`).
`rescue/victims.py`: `Registry.apply` (evidence), `settle` (verdicts), `distinct` (one report per person).
Negative reports: `Simulator._sense`.

8 Deciding where to go

This chapter answers the question you asked most directly: *how does a robot in an unknown building choose its next target, and how do coverage patterns, team strategies and route planners fit together?* The short answer: decisions are made in **layers**, and each layer answers one question.

8.1 Goals: the robot's intermediate targets

A robot never plans "search the whole building". It always has exactly one **goal**: a cell to reach and a reason to reach it. Reaching goals one after another is how the whole building gets covered. There are eight kinds:

Goal	What it means	Where it comes from
frontier	drive to the edge of the searched area and look around (360° scan on arrival)	frontier pieces on the robot's map (chapter 6)
sweep	the next waypoint of my lawnmower or spiral pattern	the robot's coverage pattern (8.4)
verify	go 0.7–2.2 m from an uncertain sighting and take a close look	open candidates in the belief filter (7.5)
home	return to the base: battery low, all known victims found, or nothing left	special rules (8.3)
report	drive back into radio range to deliver a find (limited-radio missions)	special rules
transit	partition strategy: head for my zone through unexplored space	the partition strategy
probe	drive into a cell the sensors cannot resolve (bumper decides)	chapter 6.5, 10.5
backtrack	back off along my own trail out of a dead end	dead-end recovery (10.6)

The distance field

To compare goals a robot must know how far each one is *by route*, not in a straight line (a room behind a wall may be 1 m away but 30 m to drive). So at each decision the robot runs **one Dijkstra search from its own cell to every cell** of its map (a "distance field", `mapping.dijkstra`). One search gives the route length to all goals at once, which is why goal selection always uses Dijkstra, whatever route planner is chosen. For pattern waypoints, which may lie in unmapped space, a second, *optimistic* field treats unknown cells as passable (the free-space assumption).

When does a robot decide?

Deciding is not free, and changing one's mind every second makes robots dither. A robot chooses a new goal only when:

- it has no goal, or has just reached it (and finished its scan or close look);
- its goal became invalid: the frontier cell was searched by a teammate, the sighting was confirmed or rejected, the waypoint turned out to be inside an obstacle;
- a new obstacle appeared on the next 20 cells of its route, or it has no route to its goal;
- its battery says it must go home;
- 12 seconds have passed since its last decision (a periodic re-check, because the map keeps changing).

8.2 The decision hierarchy

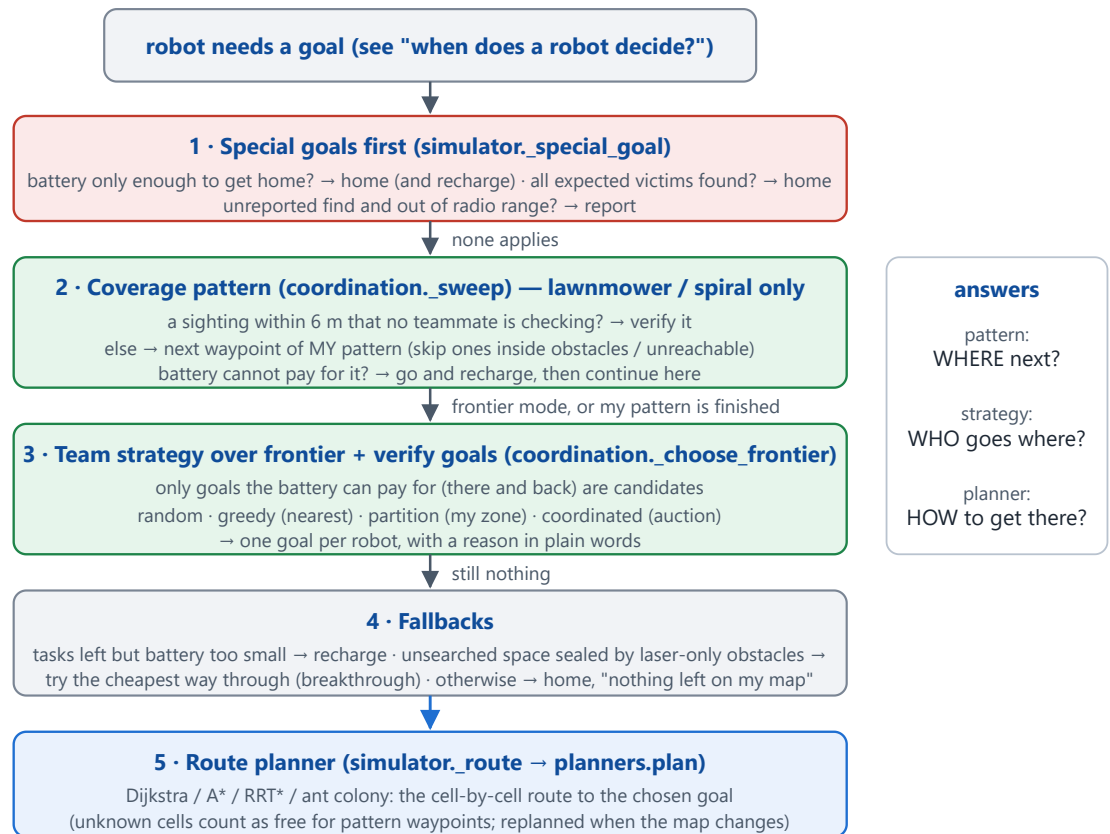


Figure 8.1 How one goal is chosen. The layers are tried top to bottom. The first layer that produces a goal wins, and the route planner then turns that goal into a route. This is `Simulator._decide`, which calls `coordination.choose`.

IN SIMPLE WORDS

First the robot asks "is there an emergency?" (battery, all found, news to deliver). If not, it asks its **coverage pattern** what to do next. With frontier exploration there is no fixed pattern, so it goes straight to the **team strategy**, which picks among the frontiers and sightings while taking teammates into account. Whatever is chosen, the **route planner** then works out the path. Finally the motion layer drives it, one cell at a time (chapter 10).

8.3 A robot's life: the state machine

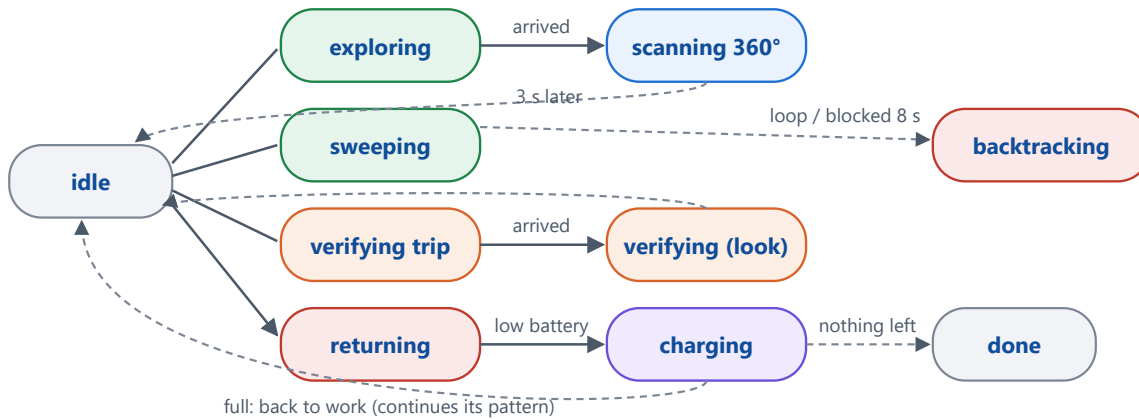


Figure 8.2 The states a robot moves through (`RobotAgent.state`). Solid arrows: choosing a goal of that kind. Dashed: what happens after. The dashboard's Robots tab shows each robot's state and its reason, e.g. "following waypoint 12 of 68 of my spiral, 3.5 m away".

8.4 Coverage patterns: where next?

A **coverage pattern** is the robot's plan for sweeping space. The project offers the three classic ones.

Frontier-based exploration (default)

No fixed path. The robot drives to a frontier (the edge of the searched area), makes a 360° camera scan on arrival (four 90° turns), and picks the next frontier. *Which* frontier is decided by the team strategy (8.5). Frontier exploration adapts to any layout, which is why it suits a building full of walls and doors.

Lawnmower (boustrophedon)

The word means "the way an ox turns when ploughing": straight lanes, alternating direction. The building's rectangle (its outline is known to rescuers; its inside is not) is split into one equal rectangle per robot. For n robots the grid is $r \times c$ with $r \cdot c = n$, choosing the factorisation whose rectangles are the most square. Then each robot is matched to a rectangle so that the *total* drive to all pattern starts is smallest. That matching is the assignment problem, solved exactly with the **Hungarian method** (`scipy.optimize.linear_sum_assignment`).

Lane spacing. A forward camera with range R and field of view F sees a cell at sideways offset e once the robot is at forward distance $f \geq e$ (inside the cone) and $\sqrt{f^2 + e^2} \leq R$. So the band swept while driving has half-width $R \cdot \sin(F/2)$. With 25 % overlap between neighbouring lanes:

$$w = 2 \cdot R \cdot \sin(F/2) \cdot (1 - 0.25) = 2 \cdot 5 \text{ m} \cdot \sin 45^\circ \cdot 0.75 \approx 5.3 \text{ m}$$

Lanes run **parallel to the rectangle's longer side**. That gives the fewest lanes and so the fewest turns, which are the costly part of a sweep (Huang, 2001). With D the width across the lanes:

$$K = \text{ceil}(D / w) \text{ lanes, lane } k \text{ at } a_0 + (k + \tfrac{1}{2}) \cdot D / K, \quad k = 0 \dots K - 1$$

So neighbouring lanes are $D/K \leq w$ apart and the outer lanes lie at most $w/2$ from the edge: nothing is left between lanes. Even lanes run one way, odd lanes the other, and lane ends stay 1 m from the region's edge.

Spiral (outside in)

From a corner, the robot drives the top edge, then the right edge, the bottom edge and the left edge. Each edge moves inwards by one spacing once it has been driven, until two opposite edges cross (the "spiral matrix" order). The passes lie on the same lines as the lawnmower's, so the same spacing guarantee holds.

From pattern to waypoints to routes

Both patterns start in the region corner nearest to the robot (the pattern is mirrored to put it there). The path is cut into **waypoints at most 1.5 m apart**. Each waypoint is a *sweep goal* that the route planner connects to the previous one. So the robot follows the pattern closely, and when an obstacle sits on a lane, the planner routes around it and returns to the lane. A waypoint found **inside an obstacle**, or **unreachable** even through unmapped space, is skipped (a red × on the dashboard). Sweep waypoints get no 360° scan, because the lane spacing already guarantees coverage. When a robot's pattern is done, it searches whatever the patterns missed (the shadows behind obstacles) by frontier exploration, so every mode ends only when nothing reachable is left unsearched.

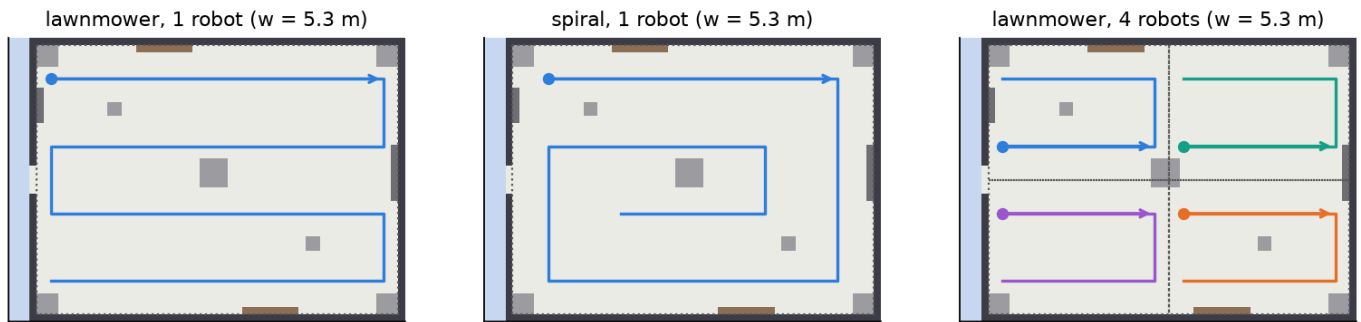


Figure 8.3 Planned patterns on the open test ground. One robot: 4 horizontal lanes, or 2 spiral rings. Four robots: the hall splits into a 2×2 grid and each robot sweeps its own quarter (2 lanes each). Dots mark where each pattern starts; arrows show the first direction.

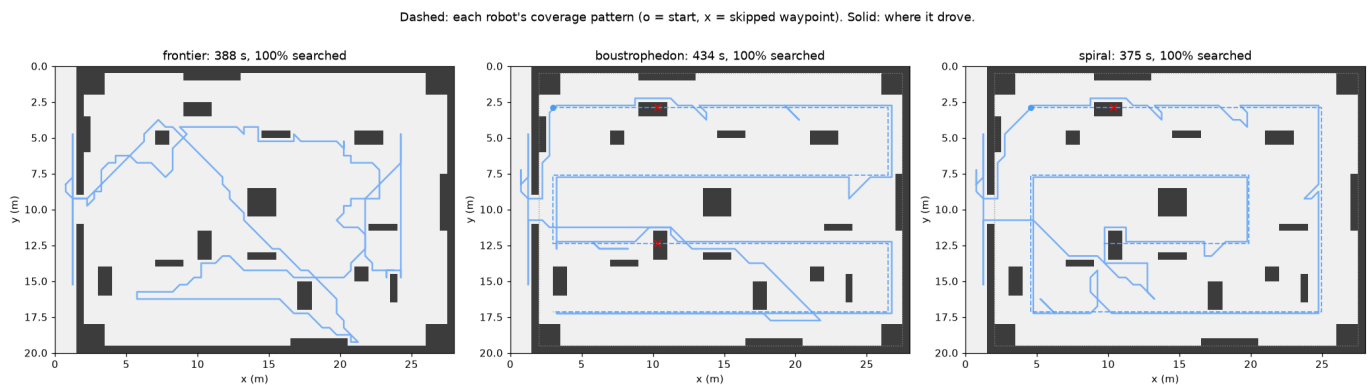


Figure 8.4 What really happened (output of `scripts/check_algorithms.py`, 1 robot): dashed = planned pattern, solid = the track driven. The lawnmower robot reached 75 of 77 waypoints (2 lay on a victim) and stayed within 0.21 m of its lanes on average (a cell is 0.5 m). The short spurs are trips to check a nearby sighting; the long diagonals at the end are the frontier mop-up and the drive home.

8.5 Team strategies: who goes where?

Strategy	How a goal is chosen	Strength	Weakness
random	any reachable goal at random	a baseline to compare against	wanders, overlaps
greedy	each robot takes its nearest goal, ignoring teammates (Yamauchi 1998)	simple, no communication needed	robots bunch up on the same frontier
partition	the building is split into one zone per robot; a robot only works in its own zone, heading there through unexplored space	no overlap at all	a robot in a small zone idles while a big zone is unfinished
coordinated	sequential auction on utility = value – 0.5 × distance, with value cut near teammates' goals (Burgard et al. 2005)	spreads the team out, one robot per sighting	needs communication

The auction, step by step

All robots that need a goal take part. Every goal has a **value** (frontier: its number of cells; sighting: $30 \times$ the current belief that it is a person) and a **distance** for each robot. Then:

```
utility(robot, goal) = value - 0.5 × distance_m           (1 m of driving costs half a point)
repeat until every robot has a goal:
  pick the (robot, goal) pair with the highest utility → that robot wins that goal
  for every other goal g, as seen by the remaining robots:
    if g is within 2 m of a won goal: g is taken (never two robots to one spot)
    if g is within 4 m of a won goal: value(g) × (distance to it / 4 m)  (a teammate will see
most of it)
    a sighting already won by someone: taken (one robot checks each sighting)
```

Robots whose teammates hold goals from earlier decisions see those goals as already won, so a robot deciding alone still avoids its teammates' targets.

WORKED EXAMPLE: WHY THE AUCTION BEATS GREEDY

goal	value (cells)	R0 distance	R1 distance	R0 utility	R1 utility
F1 (small room nearby)	12	4 m	3 m	$12 - 2 = 10$	$12 - 1.5 = 10.5$
F2 (big hall)	20	16 m	6 m	$20 - 8 = 12$	$20 - 3 = 17$
F3 (corner, 2.5 m from F1)	8	5 m	7 m	$8 - 2.5 = 5.5$	$8 - 3.5 = 4.5$

Greedy: R0's nearest goal is F1 (4 m), and R1's nearest is also F1 (3 m). Both drive to the same small room and the big hall waits.

Auction: the best pair overall is R1 → F2 (17). F2 is now taken for R0, whose best remaining goal is F1 (10). The team splits up and both areas are searched at once. A third robot would then see F3's value cut from 8 to $8 \times 2.5/4 = 5$, because R0 at F1 will already see most of that corner.

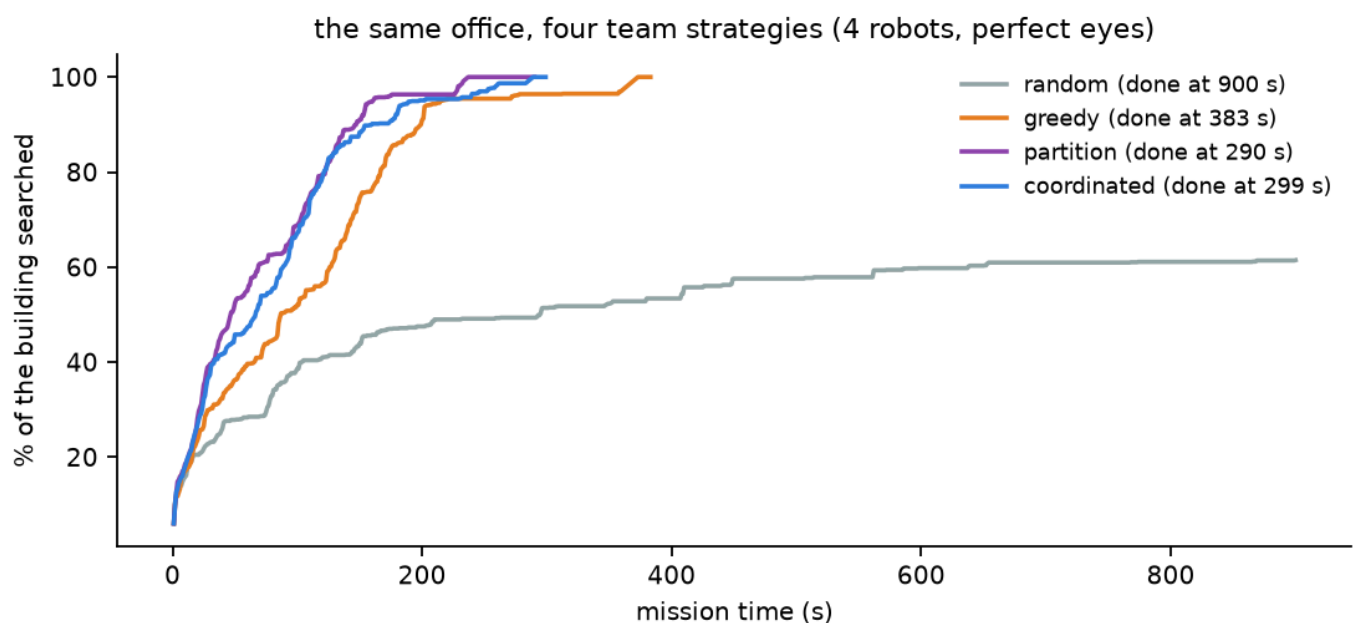


Figure 8.5 The same office searched with each strategy (4 robots, perfect eyes, so only teamwork differs). Partition (done at 290 s) and the coordinated auction (299 s) spread the team out and finish first. Greedy robots bunch up on the same frontiers and need 383 s. Random wanders and has searched only 61 % when the 900 s limit ends the mission. One building is only an example. Over 24 buildings (chapter 14), partition is also fastest on average and drives least, with the auction close behind; greedy and random trail.

8.6 How the three layers combine

Any pattern works with any strategy and any planner, because each answers a different question:

You choose	What it changes	What it does <i>not</i> change
Coverage = frontier	goals come from the frontier; the strategy chooses among them all mission long	how routes are planned
Coverage = lawnmower / spiral	each robot follows its own pattern first; the strategy is used only for the mop-up afterwards (partition then uses the pattern rectangles as zones)	how routes are planned
Strategy	which robot takes which frontier or sighting	the route to it
Path planner	the cell-by-cell route from waypoint to waypoint or to a frontier	which goal is chosen (always from one Dijkstra distance field)

INTERVIEW TIP: "HOW DO COVERAGE, STRATEGY AND PLANNING INTERACT WITH A SPIRAL?"

"The spiral decides *where*: a list of waypoints 1.5 m apart along shrinking rectangular rings in the robot's own region. Before each waypoint the robot checks for emergencies and for sightings within 6 m. The route planner, whichever one is selected, connects consecutive waypoints on the robot's map, assuming unmapped cells are free and replanning when an obstacle appears. After the spiral, the team strategy takes over for the mop-up. The three layers are independent, so I can swap any one of them and compare fairly."

9 Route planning

Once a goal is chosen, the robot needs a route: a chain of neighbouring free cells from where it stands to the goal. The project implements four planners that obey exactly the same rules (free cells only, 8 moves, straight 1 / diagonal 1.4142, no corner cutting), so their routes can be compared fairly.

9.1 The map as a graph

A planner sees the map as a **graph**: every free cell is a *node*, and every allowed move to a neighbour is an *edge* with a cost (1 or 1.4142, plus an optional extra cost per cell such as the revisit penalty of chapter 10). Finding the cheapest route is then the classic **shortest-path problem**.

9.2 Dijkstra (1959)

Dijkstra's algorithm grows the set of cells whose shortest distance is known, always taking the closest unfinished cell next. It uses a **priority queue** (a heap) ordered by *g*, the cost from the start:

```
g[start] = 0; every other g = ∞; heap = [(0, start)]
while heap not empty:
    (g_c, c) = pop the entry with the smallest g          # c's distance is now final
    if c is the goal: stop, and follow parent[] links back to the start
    for each allowed move c → n with cost k:
        if g_c + k < g[n]: g[n] = g_c + k; parent[n] = c; push (g[n], n)
```

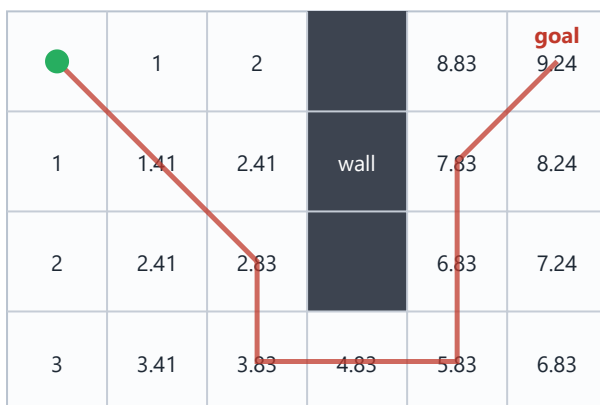


Figure 9.1 Dijkstra's distances (in cells) from the green start, computed by the project's own `dijkstra`. The route (red) goes round the wall. At (2,2) it cannot cut diagonally to (3,3), because (3,2) is wall, so it steps down first. Route cost 9.24 cells = 4.6 m.

Why it is correct: all step costs are positive, so when a cell is popped with the smallest *g*, no later path can reach it more cheaply. Its distance is final. Dijkstra therefore always returns the **optimal** route.

Cost: with a binary heap, $O((V + E) \log V)$. On a grid each cell has at most 8 edges, so this is about $O(V \log V)$ for *V* cells. For the 2,240-cell floor that is a few milliseconds.

Why the project uses it twice: to *choose* a goal, the robot needs the route distance to *every* candidate. One Dijkstra run from the robot, without a goal, gives exactly that (the "distance field"). With the Dijkstra planner selected, the route to the chosen goal is simply read back from that same field, so no second search is needed.

9.3 A* (Hart, Nilsson & Raphael, 1968)

A* is Dijkstra with a sense of direction. It orders the queue by $f = g + h$, where $h(n)$ is an **estimate of the remaining cost** to the goal. The project uses the **octile distance**, the exact cost on an empty 8-connected grid:

$$h(n) = \max(dx, dy) + (\sqrt{2} - 1) \cdot \min(dx, dy)$$

This *h* is **admissible** (it never overestimates, because walls can only make routes longer) and **consistent** (it never drops by more than the cost of one step). Those two properties guarantee that A* returns the **same optimal route**

length as Dijkstra while expanding far fewer cells, because it stops wasting effort on cells that lead away from the goal. Ties on f are broken in favour of the larger g (the deeper node), which reaches the goal sooner. In Figure 9.1, A* expands 16 cells against Dijkstra's 21. On 40 random routes on the test ground it expands 87 cells on average against 974, 1.1 ms against 9 ms.

IN SIMPLE WORDS

Dijkstra spreads out like a ripple in a pond until it touches the goal. A* is a ripple that is pulled towards the goal: it still checks every place that *could* be on a shortest route, but ignores places that obviously are not.

9.4 RRT* (Karaman & Frazzoli, 2011)

Sampling-based planners do not search cell by cell. They grow a **tree of straight line segments** by throwing random points into the free space. That makes them the standard choice for robot arms and drones in continuous, high-dimensional spaces. The project's version combines three well-known improvements:

1. **Two trees** grow, one from the robot and one from the goal, and try to connect after every new point (the RRT-Connect idea, Kuffner & LaValle 2000).
2. **RRT* rules:** a new point is joined through whichever neighbour within radius $r = \gamma \sqrt{(\log n / n)}$ gives it the cheapest cost from the root, and then nearby points are **rewired** through it if that shortens their cost. When a point gets cheaper, every point below it in the tree gets cheaper by the same amount, so the saving is propagated to its whole subtree. As samples grow, the tree's best route converges to the optimum.
3. **Informed sampling** (Gammell et al., 2014): once a route of cost c exists, new samples are drawn only inside the ellipse $|p - \text{start}| + |p - \text{goal}| \leq c$. Only there can a shorter route lie.

Every segment is checked against a fine collision mask (5 points per cell side, with a 0.3-cell safety margin at cell borders, so a line never squeezes diagonally between two obstacles). The final route is shortened along lines of sight, then drawn onto grid cells with **Bresenham's line algorithm**, whose 8-connected chain for a segment has exactly the octile length of that segment. If the route is not valid, or no route is found within 700 samples, the robot falls back to A* and the dashboard says so.

9.5 Ant Colony Optimisation (Dorigo, 1996)

Ants find short paths to food by laying **pheromone**: shorter paths are walked more often, collect more pheromone, and attract more ants. The planner imitates this with 10 virtual ants per round, for up to 12 rounds:

- At cell i , an ant picks the next cell j with probability $\propto \tau_j^\alpha \cdot \eta_j^\beta$ ($\alpha = 1$, $\beta = 4$), never revisiting a cell of its own walk and stepping back out of dead ends.
- τ_j is the pheromone on j ; $\eta_j = 1 / (1 + \text{detour})$, where **detour** = $c(i, j) + h(j) - h(i) \geq 0$ is how much longer the move makes the best route still possible (h = octile distance). A move straight towards the goal has detour 0; a step backwards has up to $2\sqrt{2}$.
- After each round the pheromone **evaporates** by $p = 30\%$, every ant that arrived lays Q/L on its path ($Q = 10$, L its length, so shorter paths lay more), and the best path so far is reinforced again (the **elitist** ant system). The colony stops when the best path has not improved for 3 rounds.

TWO CORRECTIONS MADE DURING THIS PROJECT

RRT* used to trace its lines onto the grid cell by cell, turning every slanted line into a staircase (10 right and 3 down cost 13 instead of 11.24), so its routes were 18 % too long. The ants' first heuristic, $\eta = 1/(\text{step} + \text{distance to goal})$, barely distinguished neighbouring cells far from the goal ($(41/40)^4 \approx 1.1$), so the ants mostly wandered (25 % too long, 370 ms). Both were fixed, and the numbers below are after the fix. It is a good story for an interview: measure against a known optimum, find the cause, fix it, measure again.

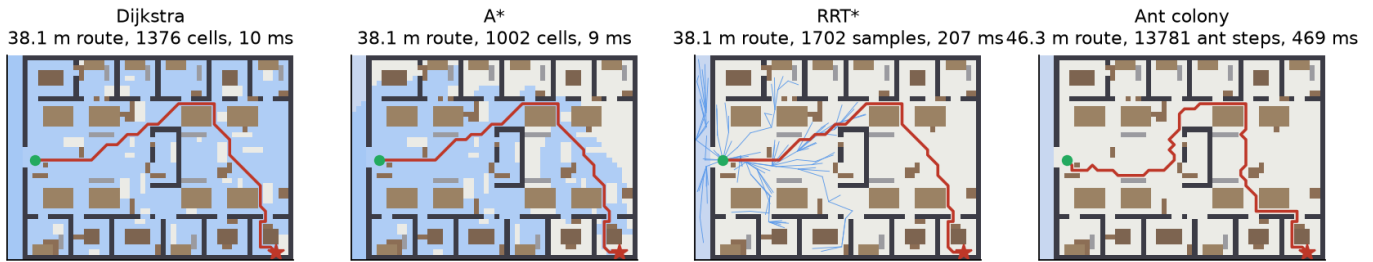


Figure 9.2 The four planners on the same route across an office, to the farthest reachable cell. Blue cells: what Dijkstra and A* examined (1,376 and 1,002 cells: A* leaves out areas that cannot lie on a shortest route). Blue lines: the two RRT* trees. Yellow: cells the ants left pheromone on. Dijkstra, A* and RRT* all find the optimal 38.1 m; this ant colony run settles on 46.3 m.

Planner (40 random routes, open test ground)	Valid routes	Length ÷ optimum (mean, worst)	Work per route	Time
Dijkstra	40 / 40	1.000, 1.000	974 cells expanded	9 ms
A*	40 / 40	1.000, 1.000	87 cells expanded	1.1 ms
RRT*	40 / 40	1.013, 1.085	116 samples	48 ms
Ant colony	40 / 40	1.011, 1.089	1,842 ant steps	100 ms

Source: `python scripts/check_algorithms.py` → `results/algorithms/check.md`. In walled buildings RRT* and the ants do worse than in open space (narrow doors are hard to hit by chance), which is why the grid planners are the sensible default for this problem.

INTERVIEW TIP: "WHY DIJKSTRA AND NOT A*?"

Don't say one is "better". Say: "Both give the optimal route on this grid. A* is faster for one point-to-point route because its admissible octile heuristic focuses the search. But to choose a goal I need distances to many candidates, and one Dijkstra run from the robot gives all of them at once. With Dijkstra as the planner, the route comes free from the same search. RRT* and ant colony are there for comparison; they suit continuous spaces and changing costs better than a small grid."

WHERE IN THE CODE

`rescue/planners.py`: `grid_search` (Dijkstra and A*, one function, heuristic on or off), `rrt_star` with `_extend`, `_propagate`, `_smooth`, `bresenham`, `_trace`; `ant_colony`; `plan` (one entry point, validation and A* fallback). `rescue/mapping.py` `dijkstra` is the distance field used for goal choice.

10 Moving and reacting

A route is a plan; the world does not always agree with it. A laser may have missed a low obstacle, a teammate may stand on the next cell, a corridor may turn out to be a dead end. This chapter explains how the robots cope, one situation at a time.

10.1 Driving a route

Each second a robot receives one second of time credit and follows its route cell by cell. A move costs $|\Delta\theta|/180^\circ/\text{s} + \text{distance}/0.5 \text{ m/s}$ (chapter 2), so a robot drives about one straight cell per second. Moves are made in up to 8 small rounds per second, so a fast robot can take several cells per second, and in each round every moving robot states the cell it wants next. Two rules resolve conflicts:

- **No two robots in one cell, and no swapping** (two robots passing through each other). If a robot's target is taken or occupied by a robot that is not moving away, it waits this round. Because a waiting robot also blocks whoever wanted *its* cell, the check is repeated until nothing changes.
- A robot on its way to check a sighting goes first.

After waiting 3 seconds, a robot drops its route and plans a new one that treats its teammates' cells as blocked: it goes around them.

10.2 Dynamic Window Approach (optional)

With *Obstacles = DWA*, the route says *where to go*, but the very next move is chosen locally by the **Dynamic Window Approach** (Fox, Burgard & Thrun, 1997). The "dynamic window" is the set of moves the robot can make right now: its 8 neighbouring cells that are free on its map, not occupied by a teammate and not a corner cut. Each is scored:

$$G = 0.45 \cdot \text{heading} + 0.25 \cdot \text{clearance} + 0.30 \cdot \text{progress} - 0.10 \cdot \text{turn}$$

Term	Meaning	Range
heading	how well the move points at the route two cells ahead: $1 - \text{angle error} /\pi$	0–1
clearance	distance from the new cell to the nearest obstacle or teammate, up to 3 cells (from a Euclidean distance transform)	0–1
progress	how much the move shortens the remaining way to the point 4 cells ahead on the route	0–1
turn	how sharply the robot must turn (smoothness)	0–1

The best move wins. If a teammate stands on the route and every alternative leads backwards, the robot waits instead. Afterwards it rejoins its route from wherever it is, or plans a new one. The effect: robots keep clear of walls and steer around each other instead of queueing. In the measurements, waits for a teammate fell from 14.5 to 3.0 per mission, for about 7 % more mission time.

10.3 What happens when...

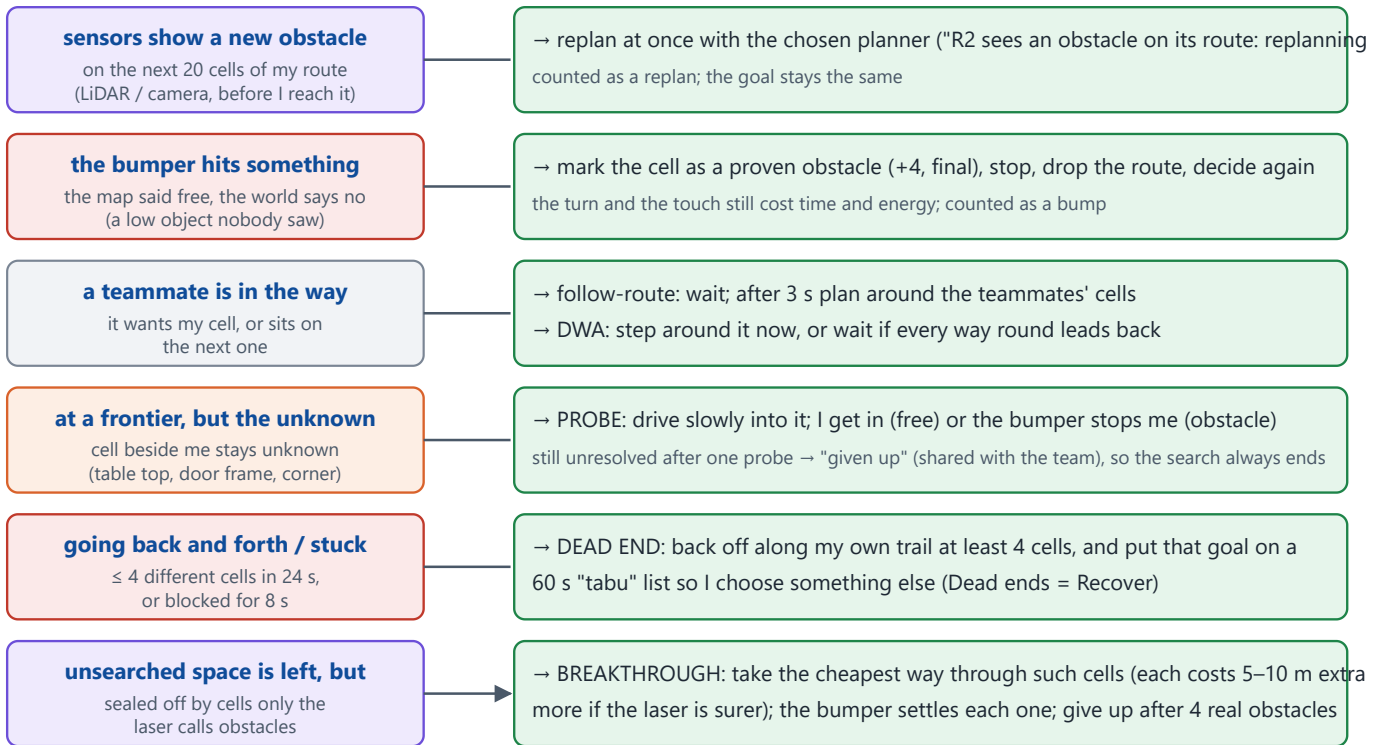


Figure 10.1 Six situations and the reaction the code takes (`Simulator._decide` , `_move` , `_probe` , `_recover` , `_breakthrough`).

10.4 Obstacles in more detail

There are three layers of defence against obstacles, from most to least preferable:

1. **Plan around what is on the map.** Routes only use FREE cells, and diagonal moves never cut corners.
2. **Replan before reaching a new obstacle.** Every second, a robot checks the next 20 cells of its route against its (just updated) map. The LiDAR sees 8 m ahead, so most obstacles appear on the map long before the robot gets there.
3. **The bumper as the last resort.** Things lower than the laser's 40 cm plane and outside the camera's view (a fallen jacket beside the robot) can be missed by both sensors. Driving into one costs a turn and a touch, then the cell is known for good.

A special case is a waypoint of a lawnmower or spiral. Waypoints are planned through unmapped space (free-space assumption). A waypoint that turns out to lie inside an obstacle is skipped, and one that becomes unreachable, even through unknown cells, is skipped too. The robot carries on with the next one.

10.5 Dead ends and loops

Why would a robot loop? Its map changes every second. A goal that looked reachable through a doorway may turn out to be blocked, and replanning can then flip between two routes. The robot therefore remembers its last 24 positions. If, while driving somewhere, it visited **at most 4 different cells** in that time, or it has been **blocked for 8 seconds**, it declares a dead end. It then backs off along its own trail to a cell at least 4 cells away (reachable, avoiding teammates), and marks the goal it was heading for as **tabu for 60 seconds**, so the next decision picks something else. With *Dead ends = Off* none of this happens, and a robot can stay stuck in a loop. The option exists to show why recovery is needed.

10.6 Revisits

Driving over ground the team has already crossed is not forbidden (it is often the only way out of a room), but with *Revisits = Avoid* it is **discouraged**: each cell costs an extra **0.3 cells per earlier team visit**, up to 5 visits, in every planner.

$$\text{cost}(\text{route}) = \Sigma \text{ step cost} + 0.3 \cdot \Sigma \min(\text{visits}(\text{cell}), 5)$$

Routes then prefer fresh ground, where the cameras see new things on the way, and robots meet less in busy corridors. Measured: moves over already-driven ground fell from 42 % to 33 %, and waits for teammates from 14.5 to 6.3 per mission, with routes only slightly longer. The *Repeat visits* layer on the dashboard paints cells redder the more often they were driven through.

INTERVIEW TIP: "CAN YOUR ROBOT GET STUCK?"

"Yes, and the system is built around that fact. Robots replan when the map changes, the bumper catches what the sensors missed, a probe resolves cells the sensors cannot, a loop/stuck detector triggers a back-off with a tabu list, and cells still unresolved after a probe are given up so the search always terminates. With recovery switched off you can watch a robot oscillate, which shows why each layer exists."

LIMITATION: GRID MOTION

Robots move between cell centres with turn-then-drive timing. There are no wheel dynamics, acceleration limits, slip or continuous velocity commands, and a robot occupies one cell (a point robot on a 0.5 m grid) rather than a body with a real footprint. A real robot would add a velocity controller (e.g. pure pursuit) and inflate obstacles by its radius.

11 Energy and batteries

A robot that runs flat in the middle of a collapsed building is a second rescue problem. So every move, every turn and every second of computing costs energy, and with a battery fitted the robots plan with their charge in mind: they refuse tasks they cannot come back from, turn home in time, and recharge.

11.1 The power model

Consumer	Model	Value (defaults)
computer	always on while working	15 W
LiDAR / cameras	if fitted	8 W / 6 W
rolling resistance	$m \cdot g \cdot C_{rr} / \eta$ per metre (25 kg, $C_{rr} = 0.05$ on debris, drivetrain $\eta = 0.6$)	20.4 J/m
motor copper losses	$k \cdot v^2$ watts = $k \cdot v$ joules per metre ($k = 12 \text{ W} \cdot \text{s}^2 / \text{m}^2$)	6 J/m at 0.5 m/s
turning on the spot	$m \cdot g \cdot \mu_t \cdot (b/2) \cdot \Delta\theta / \eta$ (tracks scrub, $\mu_t = 0.3$, track width $b = 0.4 \text{ m}$)	38.5 J per 90°

Put together: a robot with both sensors, driving at 0.5 m/s, draws about **42 W**, so **1 Wh (3,600 J) lasts about 85 seconds**. Faster driving saves base energy (less time) but costs more in motor losses, which grow with speed².

LIMITATION: WHY THE BATTERIES ON THE DASHBOARD ARE SO SMALL

A real 25 kg robot carries a few hundred Wh and works for hours. The simulated floor is only 28 × 20 m, and a mission takes minutes. So only batteries of 1–3 Wh run low within a mission, and the dashboard offers those sizes to let you watch the low-battery behaviour. The physics is the same; only the scale is shrunk.

11.2 Three battery rules

1 · before taking a task

can I get there, work there,
and still get home with margin?
no → leave it to teammates

2 · every second on the way

left ≤ 1.3 × energy of the drive
home + 5 % reserve?
yes → turn home now

3 · at the base

Recharge: dock, charge at 60 W
(1 Wh per minute), go back out
Stay: remain home for good

Guarantees

- A lawnmower or spiral robot resumes its pattern at the exact waypoint where it stopped.
- If even a full battery cannot reach anything that is left, the robot says so and stays home: no endless docking.
- With these rules no battery runs flat (the tests check it); at 0 J a robot would stop where it is.

Figure 11.1 The battery rules (`Simulator.affordable` , `_battery_low` , `_special_goal` , `_at_base` , `_battery_blocked`).

Rule 1: energy-aware task choice

For each candidate goal at route distance d , with d_{home} the route distance from that goal back to the base (from a distance field grown from the base on the robot's own map), the robot checks:

$$1.3 \cdot (e \cdot d + P \cdot (d/v + 5 \text{ s})) + 1.3 \cdot (e \cdot d_{\text{home}} + P \cdot d_{\text{home}}/v) + 5 \% \text{ of capacity} \leq \text{energy left}$$

where $e = 26.4 \text{ J/m}$ is the driving energy per metre, $P = 29 \text{ W}$ the base power, $v = 0.5 \text{ m/s}$, and 5 s the time for a scan at the goal. The 1.3 is a safety factor for detours and turns.

WORKED EXAMPLE

A robot with a 1 Wh battery (3,600 J) has 60 % left: 2,160 J. A frontier is 6 m away, and from there the base is 12 m away.

- Going there and scanning: $1.3 \times (26.4 \times 6 + 29 \times (12 \text{ s} + 5 \text{ s})) = \mathbf{847 \text{ J}}$
- Coming back: $1.3 \times (26.4 \times 12 + 29 \times 24 \text{ s}) = \mathbf{1,317 \text{ J}}$
- Reserve: $5 \% \times 3,600 = \mathbf{180 \text{ J}}$. Total needed: $\mathbf{2,344 \text{ J} > 2,160 \text{ J}}$.

So the robot does not take this frontier. If a nearer task fits, it takes that one. If none fits, it goes home to recharge: "7 task(s) left, but with 60 % battery I could not get there and back: going to recharge".

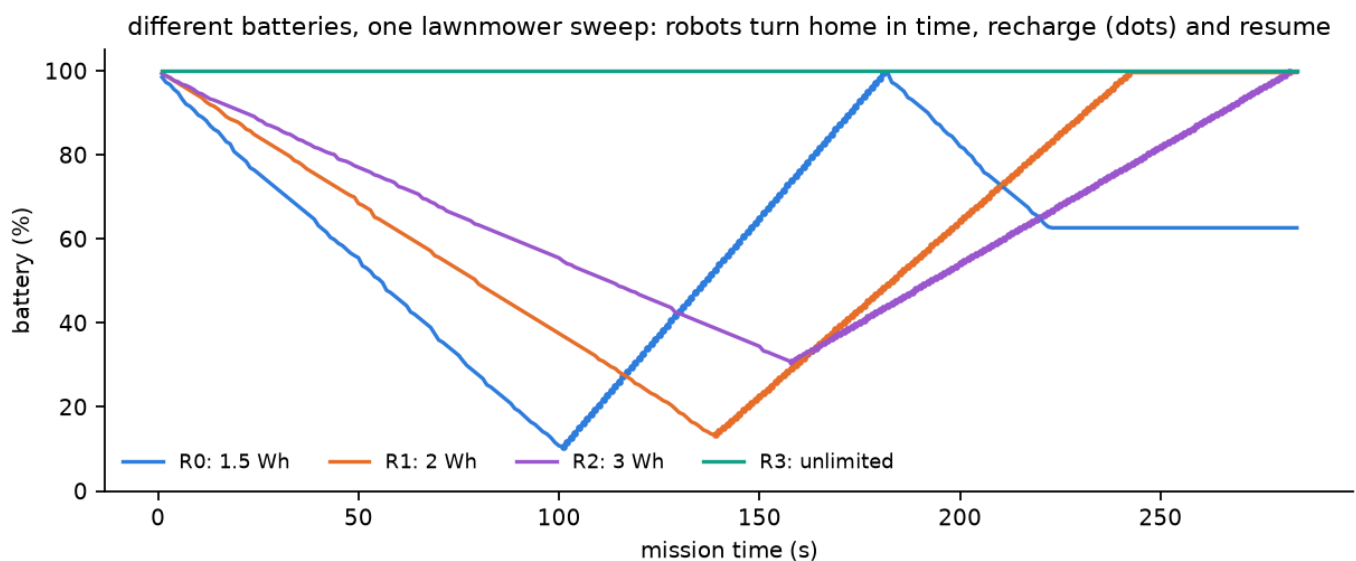


Figure 11.2 Four robots with different batteries sweep the test ground with the lawnmower. R0 (1.5 Wh) turns home at about 10 %, recharges (dotted rising line, 60 W) and goes back to its lanes. R1 (2 Wh) and R2 (3 Wh) do the same later. R3 has no battery limit. Nobody runs flat, and the whole ground is searched.

11.3 Per-robot batteries

Choosing *Battery = Different per robot* gives every robot its own capacity (1–10 Wh or unlimited). This models a mixed fleet, and it shows the energy-aware task choice at work: robots with more charge take the far tasks, and robots with less charge take near tasks or recharge. Each robot's name tag in the 3-D view shows its charge ("R1 · 62%", amber below 45 %, red below 20 %, ⚡ while charging), and the Robots tab lists capacity, charge, trips to the charger and time spent charging.

INTERVIEW TIP: "HOW DOES BATTERY AFFECT TASK ALLOCATION?"

"Each robot filters its candidate tasks by feasibility: energy to get there and do the work, plus energy to get back home, both with a 1.3 safety factor, plus a 5 % reserve, must fit in what is left. So in the auction a low-battery robot simply does not bid on far tasks, and a better-charged teammate wins them. The energy model is physical (rolling resistance, motor losses, turning, base power), and the distances come from route distance fields, not straight lines."

12 Every control on the dashboard

Start the dashboard with `python -m rescue serve` and open the address it prints. Setup is on the left, the 3-D view in the middle and the details on the right. Changing the building or the damage creates a new building. Every other change restarts the mission *in the same building*, so you can compare settings fairly. A mission waits for `Start`.

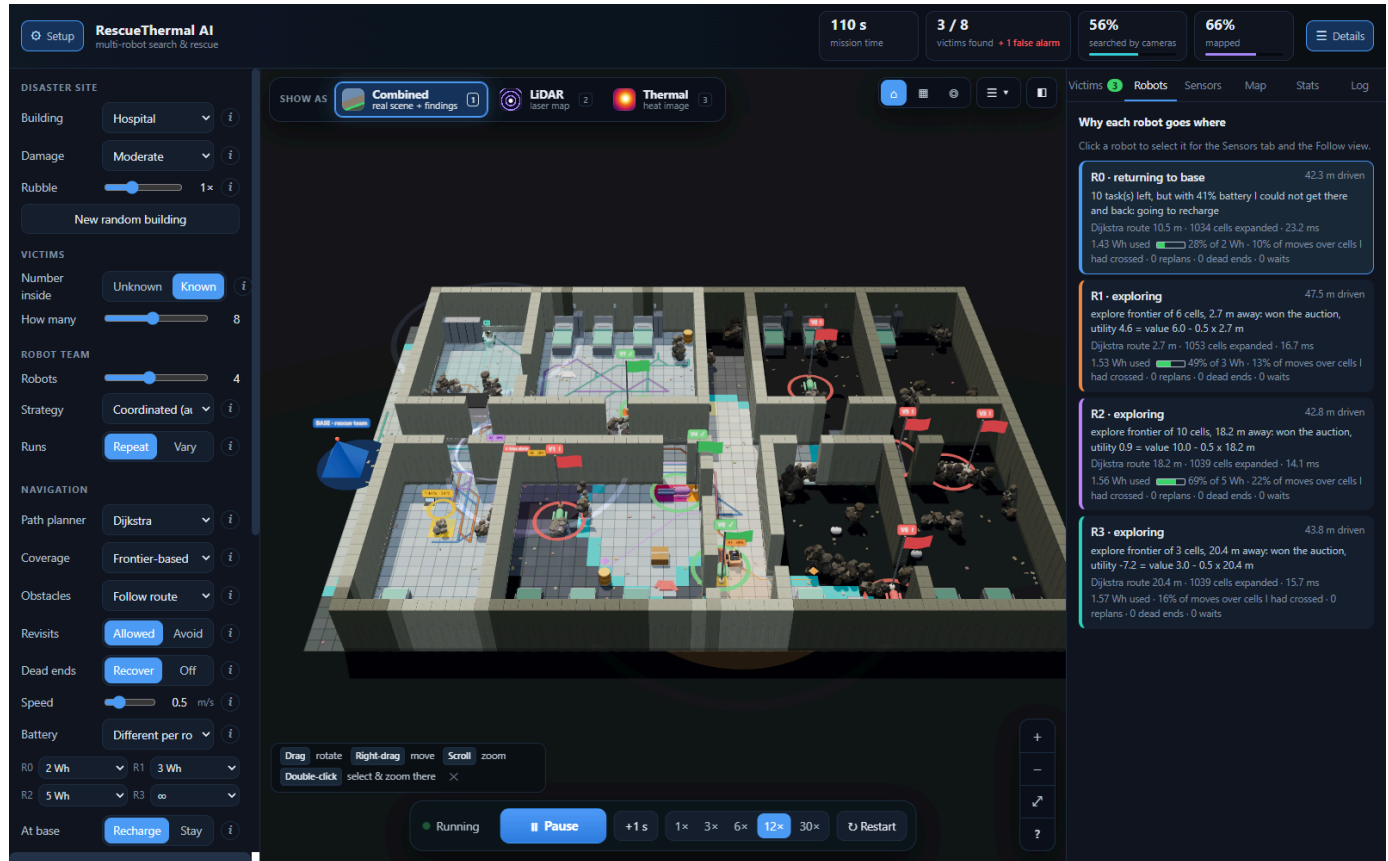


Figure 12.1 The dashboard during a hospital mission with a different battery per robot. The Robots tab (right) gives each robot's state and its reason in plain words. Here R1 "won the auction, utility 4.6 = value 6.0 - 0.5 × 2.7 m", and R0 is going home because with 41 % of its 2 Wh battery it cannot reach any remaining task and come back. Below each reason: the planner used, route length, cells expanded, time, energy and battery.

12.1 Setup panel (left)

Control	Options	What changes in the robots' behaviour
Building	office, apartments, hospital, school, parking, warehouse, open test ground	A new building of that type (chapter 3). Layouts change how useful each coverage pattern is: long corridors favour frontier exploration, open halls the lawnmower.
Damage	light, moderate, severe	More collapsed walls, rubble and buried victims (10/25/45 % of interior walls collapse; 20/40/60 % of victims partly buried; up to 20 % under thick rubble).
Rubble	0–3×	Multiplies the number of rubble piles: more obstacles, more replanning, more hidden victims.
New random building	button	Same type, new seed, so a different layout.

Number inside	Unknown / Known (+ how many, 1–16)	<i>Unknown</i> (realistic): 4–12 hidden victims, and the robots search everything. <i>Known</i> : they stop and go home once that many are confirmed (a false alarm can make them stop early).
Robots	1–8	Team size. Patterns split the building into that many rectangles; strategies spread that many robots.
Strategy	coordinated, partition, greedy, random	Who takes which frontier or sighting (8.5). With lawnmower or spiral: used for the mop-up after the patterns.
Runs	Repeat / Vary	<i>Repeat</i> : same settings, the same mission move for move. <i>Vary</i> : the same building, but new random numbers for sensor noise, tie-breaks, RRT* samples and ants, so paths differ.
Path planner	Dijkstra, A*, RRT*, ant colony	The route to each goal (chapter 9). Goals are chosen the same way whatever planner is selected.
Coverage	frontier, lawnmower, spiral	How space is swept (8.4). Lawnmower and spiral show each robot's dashed pattern and its track.
Obstacles	Follow route / DWA	Follow the route and wait for teammates, or choose each move with the Dynamic Window Approach (10.2).
Revisits	Allowed / Avoid	<i>Avoid</i> : every cell costs 0.3 extra per earlier team visit (up to 5), so routes prefer fresh ground (10.6).
Dead ends	Recover / Off	Detect loops and blockages, back off and put the goal on a 60 s tabu list (10.5). <i>Off</i> shows what happens without it.
Speed	0.2–1.5 m/s	Faster robots finish sooner, but the cameras still look once per second, and motor losses grow with speed ² .
Battery	unlimited, 10...1 Wh, different per robot	Energy-aware task choice, turning home in time, recharging (chapter 11). <i>Different per robot</i> shows one choice per robot.
At base	Recharge / Stay	After coming home on low battery: charge at 60 W and go back out, or stay home.
Sensors on the robots	Combined / Vision + Thermal / LiDAR	<i>Combined</i> : the LiDAR maps, the cameras find people. <i>Vision + Thermal</i> : no LiDAR, so the map grows only where the depth camera looks. <i>LiDAR</i> : no camera, so the building gets mapped but nobody can be found .
Camera / LiDAR range	2–8 m / 3–20 m	How far each sensor reaches. The camera range also sets the lawnmower's lane spacing.

12.2 The 3-D view (middle)

- **Show as:** *Combined* (the building as it is, plus findings), *LiDAR* (only what the lasers know: violet where beams hit), *Thermal* (only what the thermal cameras measured, hatched where nobody looked). Keys 1 2 3.
- **Camera views:** overview, top-down, follow the selected robot. Drag to rotate, right-drag to move, scroll to zoom, double-click to select and zoom.
- **Layers:** victim flags, fog over unmapped/unsearched floor, robot paths and goals, coverage patterns, where each robot has driven, camera cones, sightings and false alarms, zones, the planner search of the selected robot (cells examined, RRT* tree, ant trails), and repeat visits.
- **Legend:** what every colour means in the current picture.
- **Transport:** Start / Pause, +1 s (one step), speed 1×–30× simulated seconds per real second, Restart.

12.3 Details panel (right)

Tab	Shows
Victims	confirmed = real + false + counted twice; every victim and how they lie; every false alarm with what it really was (ground truth, simulation only)
Robots	state, reason, pattern progress, route (planner, length, work, time), energy, battery, repeat moves, replans, dead ends, waits
Sensors	the selected robot's thermal image, colour image (with the AI's heatmap and detections) and LiDAR scan (Figure 12.2)
Map	the team's victim map (what rescuers would get), exportable as JSON
Stats	curves of searched / mapped / found over time, all metrics, and Compare algorithms : run the same mission once per planner, pattern, avoidance or strategy
Log	every event in plain words: sightings, confirmations, rejections, replans, dead ends, battery decisions

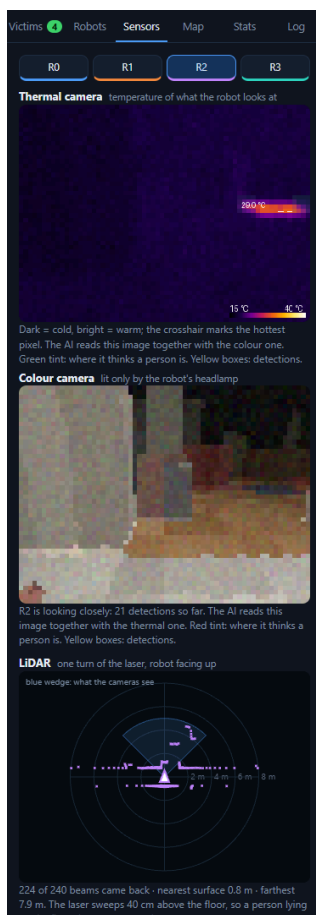


Figure 12.2 The Sensors tab: the thermal image (crosshair on the warmest pixel), the colour image as the AI sees it, and the LiDAR scan with the camera's field of view as a blue wedge.

PRESETS IN THE ADDRESS BAR

Any setup can be put in the URL, which is handy for demos and tests:

```
http://localhost:8001/?building=plain
&coverage=spiral&robots=2
&planner=astar&battery_each=1.5,3
&recharge=1&view=top&autostart=1
&speed=12&seed=42
```

Other keys: `damage`, `sensors`, `show`, `known`, `victims`, `strategy`, `avoidance`, `revisit=0.3`, `deadend=0`, `rspeed`, `battery`, `camrange`, `lidarrange`, `density`, `vary=1`, `tab`, `legend=1`.

A GOOD FIRST EXPERIMENT

Open test ground, 1 robot, Coverage = Spiral, Speed 30×, Layers: coverage patterns + where each robot has driven. Watch the robot follow the rings, skip a waypoint on a victim and check a sighting. Then switch the planner to RRT* and watch the any-angle routes, or set Battery = 1.5 Wh to watch it go home, charge and resume.

13 Software architecture

The project is about 9,000 lines: a Python package `rescue/` (simulation, AI and a small web server), a JavaScript 3-D front end that uses three.js, scripts for experiments, and 92 automated tests. Nothing needs a GPU; PyTorch is used on the CPU for the detector.

13.1 Modules

File	Lines	Responsibility
<code>config.py</code>	97	every tunable parameter in one dataclass, <code>RescueConfig</code>
<code>world.py</code>	699	procedural buildings: layouts, collapse, furniture, rubble, victims, look-alikes, fine heightfield
<code>thermal.py</code>	199	temperatures: ambient, bodies, covers, warm objects, engines; ironbow palette
<code>lidar.py</code>	108	LiDAR ray casting, noise, dropout, inverse sensor model
<code>sensors.py</code>	159	camera rendering (colour, depth, thermal, 3-D points), visible cells, line of sight
<code>mapping.py</code>	187	evidence grid, classify, merge, searched view, frontiers, Dijkstra distance field
<code>perception.py</code>	314	the CNN: dataset rendering, training, evaluation, detection
<code>victims.py</code>	172	reports, candidates, the belief filter, team verdicts, one report per person
<code>coverage.py</code>	267	lane spacing, regions, lawnmower and spiral patterns, waypoints, Hungarian assignment
<code>coordination.py</code>	341	goals, sweep following, random / greedy / partition / auction strategies
<code>planners.py</code>	482	Dijkstra, A*, RRT*, ant colony, route validation
<code>motion.py</code>	182	kinematics, energy model, DWA, loop detection
<code>simulator.py</code>	1,104	the mission: robots, base, the one-second loop, decisions, moves, batteries, recovery, scoring
<code>metrics.py</code>	163	everything measured against the ground truth
<code>server.py</code>	610	HTTP server: sessions, JSON state for the page, algorithm comparison jobs
<code>__main__.py</code>	235	command line: <code>serve</code> , <code>simulate</code> , <code>train</code> , <code>benchmark</code>
<code>web/js/*.js</code>	2,300	the dashboard: scene, building meshes, robots, floor painting, panels, charts, controls

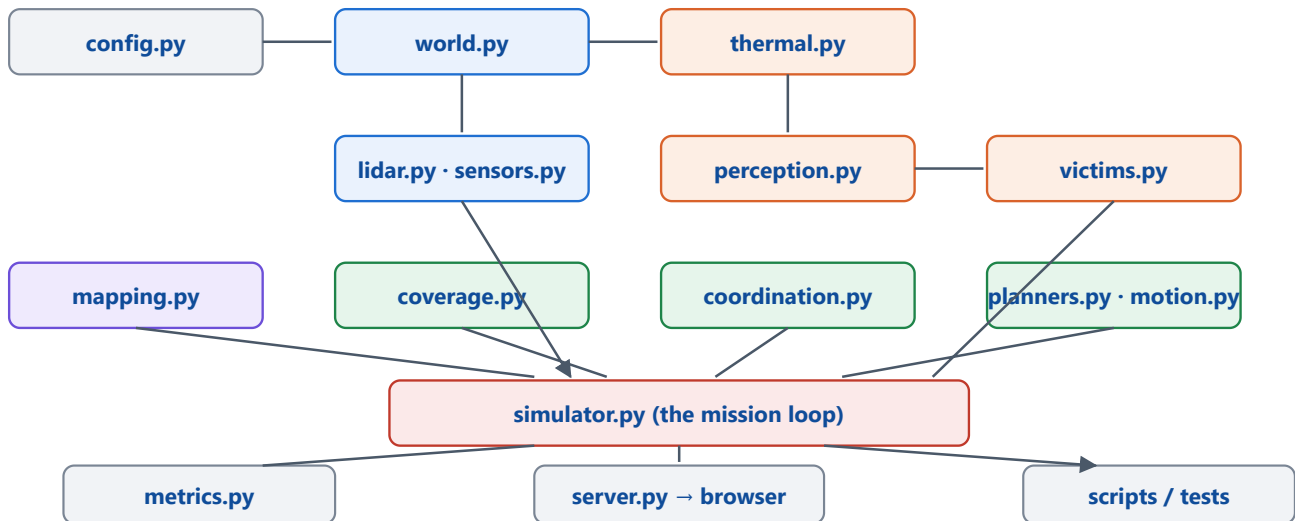


Figure 13.1 Who uses whom. The simulator owns the mission and calls into every module. Everything above it knows nothing about the web page, so the same simulation runs in the dashboard, the command line, the tests and the experiment scripts.

13.2 One step in code

```

Simulator.step():
    for each robot: _sense(r)      # lidar.scan → mapping.apply_lidar; sensors.visible_cells →
    apply_camera;                  # sensors.render → detector.detect → _report (into the robot's
                                   # who can talk to whom (radio range, walls)
    outbox)                        # merge maps; apply new reports in id order; registry.settle →
    _groups = groups()            # distance fields; special goals; coordination.choose; _route →
    _communicate()                # scans, looks, charging; conflict-free moves; energy; dead-end
    verdicts                      # ground-truth bookkeeping for the metrics
    _decide()
    planners.plan
    _act()
    recovery
    _score()

```

13.3 The dashboard: server and browser

The server is a small standard-library HTTP server (`ThreadingHTTPServer`). Each browser gets its own **session** (its own simulator), so two people can run different missions at once. The browser asks for batches of steps and draws the result:

Endpoint	Returns
<code>/api/reset?building=...&robots=...</code>	a new mission with those settings; the world (walls, furniture, zones, patterns) as JSON
<code>/api/step?n=K</code>	advance K seconds; the new state: robots (position, goal, route, reason, plan, battery, pattern, track), team map, searched cells, thermal map, frontiers, candidates, metrics, log
<code>/api/camera</code>	the selected robot's colour / thermal images and heatmap as PNG
<code>/api/compare</code>	starts a background job that runs the same mission once per option (planner, pattern, avoidance, strategy)
<code>/api/victim_map ,</code> <code>/api/reveal</code>	the exportable victim map; reveal the ground truth when the count is unknown

Large grids are sent compactly: the team map as one character per cell, the searched layer as bits, the thermal map as bytes. The front end (plain ES modules, no build step) builds the building once with instanced meshes, then each step repaints the floor texture (map, searched fog, frontiers, patterns, tracks) and moves the robots, gliding between states for smooth animation.

13.4 Tests and scripts

92 tests (`pytest` , about 6 minutes) check the maths and the behaviour: A* returns Dijkstra's optimal length while expanding fewer cells; the octile heuristic never overestimates; every planner returns a valid route never shorter than the optimum; Bresenham lines have octile length; RRT* tree costs stay consistent after rewiring; lanes are at most w apart; spirals go outside-in and always turn the same way; robots really drive their patterns (every reached waypoint was driven through, the median distance from the pattern is at most 0.36 m); every navigation option searches 100 % of the building; robots never drive faster than their speed; the same settings give the same mission; batteries never run flat and recharging resumes the pattern; robots sharing their reports always agree; no object ever cuts off part of a building; and the dashboard's numbers add up.

Command	Does
<code>python -m rescue serve</code>	start the dashboard
<code>python -m rescue simulate --strategy coordinated</code>	run one mission on the command line and print its metrics
<code>python -m rescue train</code>	render 20,000 labelled images and train the three detectors (about 50 minutes each on a CPU)
<code>python -m rescue benchmark --seeds 8</code>	compare strategies and vision modes over many buildings
<code>python scripts/check_algorithms.py</code>	planners and coverage patterns on the test ground → <code>results/algorithms/</code>
<code>python scripts/reliability.py</code>	every confirmation checked against the truth → <code>results/reliability/report.md</code>
<code>python -m pytest</code>	the 92 tests

INTERVIEW TIP: "HOW IS YOUR CODE STRUCTURED?"

"Layered, like the robot's own architecture: world and sensors, then maps and perception, then decisions and motion, with one simulator class owning the loop. The simulation has no dependency on the UI, so the same code runs the web dashboard, the CLI, experiments and tests. All parameters live in one config dataclass, and every random choice comes from seeded generators, so any mission can be reproduced exactly."

14 How good is it? Measuring

A simulation is only useful if you can measure it honestly. The robots never see the ground truth, but the scoring code does: after every mission it checks every confirmed "person" against where people really are.

14.1 What is measured

Metric	Meaning
searched (coverage)	share of the reachable floor that some camera has looked at
mapped	share of the reachable floor on the team map (LiDAR or camera)
found / recall	victims with a confirmed report within 1.5 m (one report per person); recall on <i>detectable</i> victims excludes those under thick rubble
precision, false alarms	share of confirmed reports that were real people; the rest split into warm objects, look-alikes, nothing, and people counted twice
time to first / 50 % / all victims	seconds until those were confirmed; time 90 % searched / mapped
distance, energy, recharges	team totals
redundancy, repeat moves	how many cameras looked at each cell on average; share of moves into cells already driven through
waits, replans, dead ends, bumps	how often robots had to react (chapter 10)
planning time and work	ms and cells / samples / ant steps per route

14.2 Reliability of the victim search (current rule)

`scripts/reliability.py` runs 12 missions per sensor set (six damaged building types × 2 seeds, 4 robots, 10 victims, 12 look-alikes, 6 warm objects) and classifies every confirmation and every rejection by what is really there:

Sensors	Victims found / findable	Confirmed	Real	Counted twice	False alarms	Precision	Seconds robots disagreed
colour camera	80 / 102	93	79	1	13	85 %	0
thermal camera	99 / 102	118	98	4	16	83 %	0
colour + thermal	100 / 102	113	99	2	12	88 %	0

What were the false alarms? With both cameras: a **dog** (5), **sun-warmed rubble** (3), nothing there (2), a jacket and a hot-water leak. With the thermal camera alone: mostly sun-warmed rubble (8). With the colour camera alone: mostly **bags** (8) and laptops (2). The colour camera misses **all 18** people under thin debris; the thermal and fused detectors find every one of them, because heat leaks through. No sensor finds the 18 people under thick rubble, as designed.

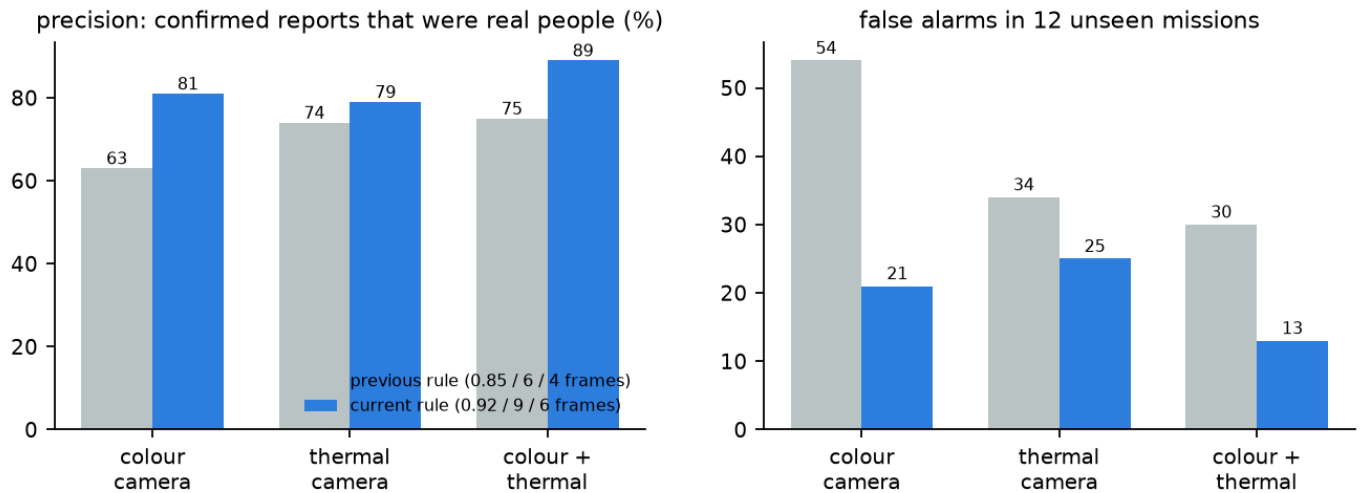


Figure 14.1 The confirmation rule compared on 12 *unseen* missions per sensor set (seeds 400–401, not used for tuning). The stricter rule (detection 0.92, evidence 9, 6 frames) more than halves the false alarms for 1–3 fewer confirmed victims per 108. Those stay on the victim map as unconfirmed sightings.

HOW TO READ PRECISION

88 % precision means roughly 1 in 9 people the fused team reports is not a person. For a real rescue that is a useful, honest number: the rescuers get a map where most flags are people, each with its measured temperature and how many robots saw it, and every flag is checked by a human before anyone digs.

14.3 Team strategies over 24 buildings

From the project's benchmark (4 buildings of each of the six types, seeds 200+, 4 robots, 900 s limit, perfect eyes so only teamwork is compared):

Strategy	50 % of victims found	all found	90 % searched	distance driven	redundancy
random	267 s	617 s (11/24 runs)	553 s	1,797 m	3.36
greedy	82 s	165 s	147 s	496 m	2.59
partition	55 s	135 s	117 s	359 m	1.87
coordinated	64 s	133 s	126 s	470 m	2.48

Partition and the auction are clearly better than greedy and far better than random. Partition drives least, because zones never overlap. The auction is as fast to find everyone and keeps working when zones are unequal. With perfect eyes all four strategies eventually find 100 % of detectable victims except random (86 %).

These benchmark tables in the README were measured before the confirmation-rule change; the navigation comparison (perfect eyes) is not affected by it.

14.4 Navigation checks on the test ground

`scripts/check_algorithms.py` verifies the algorithms against known answers (chapter 9 has the planner table): all four planners return valid routes, Dijkstra and A* are exactly optimal, RRT* and the ants are within about 1 % on average. With 1 robot the lawnmower sweeps 4 lanes (108 m), reaches 75 of 77 waypoints, keeps within 0.21 m of its lanes on average and has searched 97.3 % when done; the spiral (2 rings, 95 m) reaches 67 of 68 waypoints, 0.29 m, 97.5 %. The frontier mop-up finishes the last few percent behind obstacles.

15 Honest limitations

Every simulation simplifies. Knowing exactly where this one does, and what a real system would need instead, is a sign of understanding, and interviewers often ask. Here is the complete list.

Simplification	What the project does	What a real robot would need
Localisation	each robot knows its exact cell	SLAM or localisation: wheel odometry + IMU + scan matching (e.g. Cartographer, SLAM Toolbox), with position uncertainty in every map update
Grid motion	point robot moving between 0.5 m cells, turn-then-drive timing	a footprint, obstacle inflation by the robot's radius, a velocity controller (pure pursuit, MPC), acceleration limits, slip on rubble
2.5-D world, one floor	a heightfield: nothing overhangs, no stairs	3-D maps (voxels, OctoMap), traversability of slopes and debris, multi-floor navigation
Sensors	ray-cast LiDAR and camera with noise, dropout, haze, blur and drift	reflective and transparent surfaces, dust storms, motion blur, rolling shutter, calibration between sensors
Thermal physics	assigned surface temperatures, exponential heat leak through rubble	emissivity and reflections, heat conduction over time, thermal crossover at different times of day, radiometric calibration
Detector training	trained only on simulator images (the network has never seen a real photo)	real or domain-randomised data; sim-to-real transfer is a research problem in itself
Victims	lie still, 1.75 m bodies, no sound, no movement	moving or calling people (motion and audio cues), children, different postures
Communication	unlimited on the dashboard; a simple range model with walls (35 %) on the command line	packet loss, bandwidth limits, mesh relaying, map compression
Coordination	the auction is computed centrally, each second, for robots that are deciding	distributed bidding with messages and delays (e.g. a market-based protocol), robustness to lost robots
Batteries	tiny capacities so that they matter on a 28 × 20 m floor; perfect charge measurement; a 60 W charger at the base	realistic packs (hundreds of Wh), voltage sag, temperature effects, docking manoeuvres
Dynamic world	the building does not change during a mission	aftershocks, secondary collapse, moving rescuers: maps must forget old evidence

WHY THESE SIMPLIFICATIONS ARE REASONABLE

The project studies *decisions*: how to explore, how to split work, how to plan and recover, how to turn noisy detections into reliable reports. Each simplification removes a problem that has its own large literature (SLAM, motion control, sim-to-real) without changing the decision logic that sits on top of it. A useful sentence for an interview is: "The navigation and coordination logic would run unchanged on top of a real localisation and control stack; what would change is where the map and the pose come from."

15.1 What is deliberately realistic

- Robots know nothing about the inside of the building: they discover it with their own noisy sensors.
- The LiDAR is blind below its scan plane, and the camera is blocked by tall furniture. Both limits shape the search.
- Some victims cannot be found by any camera (thick rubble), and some look-alikes fool each sensor. Results report both instead of hiding them.
- Every move costs time according to its length and turn, and energy according to a physical model.
- Every result is checked against the ground truth on missions not used for tuning.

15.2 Natural next steps

1. Add pose noise and a simple scan-matching localiser, and measure how map quality and search time degrade.
2. Replace the central auction with message-based bidding and a lossy radio.
3. Add a sound sensor (calling victims) and fuse it with the thermal evidence in the belief filter.
4. Inflate obstacles by the robot's radius and follow routes with pure pursuit, for continuous motion.
5. Train the detector with domain randomisation, then test it on real thermal images.

16 Interview questions and answers

Sixty-four questions an interviewer could ask about this project, with answers you can give in your own words. Each answer points to the chapter with the details.

The big picture

Q. Describe the project in two sentences.

A. A simulation of a team of ground robots that search an unknown, damaged building for people using LiDAR, a colour camera and a thermal camera. It covers the full autonomy stack: procedural worlds, sensor models, mapping, a CNN victim detector with a Bayesian confirmation filter, multi-robot task allocation, four route planners, reactive motion and energy management, all measured against ground truth. (Ch. 1)

Q. What is the main loop?

A. Once per simulated second every robot senses, detects, communicates, decides and acts. It is a closed loop: acting changes what is sensed next, and the map and plans are updated every second. (1.1)

Q. Which parts are "AI"?

A. Machine learning is used only for perception: a small fully convolutional network turns camera images into a person heatmap. Everything else is classical robotics: occupancy grids, frontier exploration, auctions, graph search, sampling-based planning, swarm optimisation and a Bayesian filter. (7, 8, 9)

Q. What does the robot know, and what does only the simulator know?

A. The robot knows the building's outline and its own position, and whatever its sensors and teammates tell it. The simulator knows the true layout and where people are, and uses that only to render sensor readings and score results. (1.1)

Q. Is the world 3-D? Why is the planner 2-D?

A. The world is 2.5-D (a heightfield) and drawn in 3-D. Ground robots move on the floor, so (x, y, heading) is enough for planning. Heights still matter because they decide what the LiDAR (40 cm plane) and camera (0.55 m) can see. Multi-floor or aerial robots would need 3-D maps. (2.3)

Q. How do you make experiments fair?

A. Seeded random generators with separate streams for building, heat, sensor noise and planning. Changing a setting reruns the same building with the same noise, and "Vary" changes only the mission's random numbers. (2.6)

Q. What was the hardest problem?

A. Good candidates: making robots with the same evidence reach the same verdict (the order-dependence bug), making the coverage patterns real sweeps instead of goal re-ranking, and the RRT* and ACO corrections found by measuring against a known optimum. (7.6, 8.4, 9.5)

Q. How big is it?

A. About 9,000 lines: a Python package, a three.js dashboard, experiment scripts and 92 tests. (13)

World and sensors

Q. How are buildings generated?

A. A shell with one entrance, a layout function per type (rooms along corridors, wards, classrooms, parking bays, racks), random collapse of interior walls, furniture, rubble random walks, then victims, look-alikes and warm objects, and finally a temperature field. Everything comes from one seed. (3.1)

Q. How do you ensure every room is reachable?

A. Every object is placed tentatively. A flood fill (BFS) from the entrance must show that the reachable area shrank by exactly the object's own cells; otherwise the placement is rejected. (3.1, step 4)

Q. How does your LiDAR work?

A. Ray casting: 240 beams, 1.5° apart, marched in 8 cm steps (refined to 1 cm) until the fine heightfield exceeds the 40 cm scan plane, up to 8 m. Gaussian range noise of 2 cm and 1 % dropouts are added. Beams passing through a cell are evidence of free space, and beams ending in it evidence of an obstacle. (4.1)

Q. Why can't the LiDAR find people?

A. A lying person is below its scan plane, so the beams pass over them. Even if a beam hit one, a distance does not say "person". LiDAR maps; cameras find people. (4.1)

Q. How is the camera image produced?

A. A pinhole model (64 × 48, 90°, tilted 14° down) ray-marched through the heightfield in 7 cm steps: depth, colour × shading × headlamp fall-off, a 3-D point per pixel, then haze, exposure and pixel noise. (4.2)

Q. How is the thermal image produced?

A. The same rays read the temperature of the surface they hit, then dust attenuation, a 0.8-pixel blur, 0.25 °C pixel noise and a 0.3 °C frame drift are applied. (4.3)

Q. How do you assign temperatures?

A. Ambient per building minus collapse cooling, a smooth drift field, cooler walls; skin 33–36 °C with possible hypothermia; clothing 4–7 °C cooler; heat through rubble falling as $e^{-d/10 \text{ cm}}$; warm look-alikes (dogs 31–36 °C, heaters 42–60 °C, engines 40–65 °C...). (5)

Q. Can the robots find someone under thick rubble?

A. No, by design: 40–80 cm of rubble lets through under 2 % of the warmth, so no camera sees anything. Those victims are counted separately as undetectable, so the results never pretend otherwise. (5.3)

Q. What is the 40 °C rule?

A. A physics check: nothing on a living person is hotter than 40 °C, so a "person" detection whose hottest pixel exceeds it is dropped (heaters, engines). It cannot remove dogs, which are in the human range. (5.5)

Q. Is the thermal data real?

A. No. It is a physically motivated synthetic sensor model. It includes the ambiguities that matter (warm look-alikes, heat blocked by rubble), but not emissivity or reflections. (4.3, 15)

Maps

Q. What is an occupancy grid?

A. A grid where every cell holds the belief that it is occupied. Here it is log-odds: beams passing through add -0.45, beams ending add +0.9, the laser saturates at ±2.5, and the camera and bumper set ±4. Cells are FREE below -0.8 and OBSTACLE above +0.8. (6.1)

Q. Why log-odds?

A. Independent evidence combines by addition in log-odds, which is Bayes' rule without multiplications, and it is numerically stable near 0 and 1. (6.1)

Q. What is the difference between "mapped" and "searched"?

A. Mapped means the robot knows whether it can drive there. Searched means a camera has looked there for people. The laser maps further than cameras search, so exploration is driven by the searched layer. (6.2)

Q. What is a frontier?

A. A searched free cell next to an unsearched cell: the boundary between known and unknown. Frontier cells are clustered, long clusters are cut into ~10-cell pieces along their main axis (SVD), and each piece becomes a goal with value = its size. (6.3)

Q. How do robots merge maps?

A. For each cell they keep the strongest single opinion, not the sum, because the same observation passed back and forth must not count twice. Proven-solid cells stay obstacles. Searched layers are ORed. (6.4)

Q. What if the sensors can never resolve a cell?

A. The robot probes: it drives slowly into the cell. It either gets in or the bumper stops it. If the cell is still unresolved after one probe, it is given up and no longer counts as a frontier, which guarantees termination. (6.5)

Q. Do you do SLAM?

A. No. Poses are exact, and the project focuses on mapping, exploration and decisions. A real deployment would add localisation, and the decision layers would run unchanged on top. (2.5, 15)

Machine learning and confirmation

Q. What model do you use?

A. A small fully convolutional CNN (about 73k weights): two conv blocks with pooling, three dilated conv layers, and a 1×1 output. It maps a 48×64 image with 4, 2 or 5 channels to a 12×16 heatmap of "part of a person" per 4×4 block. It is trained from scratch, not YOLO. (7.3)

Q. Where did the training data come from?

A. The simulator: 200 random disaster zones × 100 views = 20,000 images, biased towards victims and hard negatives, labelled automatically from per-pixel object ids. 40 unseen zones (4,000 images) are used for testing. (7.2)

Q. How did you handle class imbalance?

A. Person blocks are rare, so the BCE loss up-weights them by $\sqrt{((1-p)/p)}$, capped at $\sqrt{20}$. Without that the network could win by always predicting "nobody". (7.3)

Q. Why dilated convolutions?

A. To see a whole 1.75 m body from each output cell without further pooling. Context is what separates a person from a 0.75 m dog. (7.3)

Q. How does a pixel become a map position?

A. For the peak block of a detection, the nearest depth reading's 3-D point (what an RGB-D camera gives) is the position; with thermal, the block's hottest reading is attached. (7.4)

Q. Why not trust one detection?

A. Single frames are noisy and fooled by look-alikes. A candidate accumulates log-odds evidence (close looks count 1.0, far ones 0.55, "saw nobody" -0.9) and is confirmed only with evidence ≥ 9 , 6 separate frames and one look from ≤ 2.5 m. (7.5)

Q. How were those thresholds chosen?

A. By a reliability study checked against ground truth, then validated on unseen missions: the stricter rule raised fused precision from 75 % to 89 % for 2 fewer confirmed victims out of 108. (7.5, 14)

Q. You saw a dog confirmed and something else rejected. Is that a bug?

A. A dog being confirmed is realistic: same temperature as a person, body-like shape. What was a bug is that two robots could reach different verdicts on the same evidence, because of report order and when the verdict was taken. That was fixed: reports are applied in id order and verdicts are taken once per second after sharing. Measured disagreements went from 7 to 0. (7.6)

Q. What is "counted twice"?

A. A real person confirmed a second time more than 1.6 m from the first report, e.g. head and legs. It is an error in the count, not a phantom, and is reported separately from false alarms. (7.6)

Q. How reliable is it?

A. With both cameras, 100 of 102 findable victims found and 88 % precision; the colour camera alone misses everyone under thin debris. All numbers come from missions checked against ground truth. (14.2)

Decisions and coordination

Q. How does a robot choose its next target?

A. In layers: special goals first (battery, all found, report); then its coverage pattern (next waypoint, or a sighting within 6 m); otherwise the team strategy over frontier and verify goals it can afford; then fallbacks. The chosen goal is handed to the route planner. (8.2)

Q. How are coverage, strategy and planning related?

A. They answer different questions. The pattern says WHERE next, the strategy WHO takes which task, the planner HOW to get there. Any combination works, so each can be compared fairly. (8.6)

Q. Explain frontier-based exploration.

A. Repeatedly drive to a boundary between searched and unsearched space and look around. Every trip reveals new area, and the process ends when no reachable frontier remains (Yamauchi 1997). (8.4)

Q. How is the lawnmower's lane spacing chosen?

A. From the camera footprint: half-width $R \cdot \sin(F/2)$, with 25 % overlap, gives $w \approx 5.3$ m. Lanes run parallel to the longer side (fewest turns), $K = \text{ceil}(D/w)$ lanes at $a_0 + (k + \frac{1}{2})D/K$, so gaps never exceed w . (8.4)

Q. How do multiple robots share a lawnmower sweep?

A. The building's rectangle is split into an $r \times c$ grid of near-square rectangles ($r \cdot c = n$), and robots are matched to them by the Hungarian method to minimise total travel to the pattern starts. (8.4)

Q. What happens when a waypoint is inside an obstacle?

A. It is skipped and recorded. The route between waypoints goes around obstacles, and after the pattern, frontier exploration searches what was missed. (8.4)

Q. Explain the auction.

A. A sequential single-item auction: utility = value – $0.5 \times$ route distance. The best (robot, goal) pair wins, goals within 2 m of it become unavailable and those within 4 m lose value in proportion, and each sighting goes to one robot only. Repeat until every deciding robot has a goal. (8.5)

Q. Why does greedy perform worse?

A. Every robot takes its nearest frontier, ignoring teammates, so they bunch up on the same opening. In the benchmark greedy needs about 20–25 % longer than partition or the auction to find everyone, and drives more. (8.5, 14.3)

Q. Centralised or distributed?

A. The auction is computed centrally each second over the robots that are deciding, using goals teammates already hold. A distributed version would exchange bids by radio; the utilities would not change. (15)

Q. How does battery affect allocation?

A. Tasks are filtered by feasibility: $1.3 \times$ (energy there + 5 s of work) + $1.3 \times$ energy back + 5 % reserve must fit in what is left. Low-charge robots don't bid on far tasks, go home, recharge at 60 W and resume where they stopped. (11)

Route planning

Q. Walk me through Dijkstra.

A. Priority queue by cost from the start; pop the cheapest, and its cost is final (costs are non-negative); relax its 8 neighbours with costs 1 or $\sqrt{2}$ and no corner cutting; stop at the goal and follow parent links. $O((V+E) \log V)$. (9.2)

Q. Why is Dijkstra the default?

A. Goal selection needs route distances to many candidates, which one Dijkstra run from the robot provides. With Dijkstra as planner the route to the chosen goal comes free from that same search. (9.2)

Q. What makes A*'s heuristic admissible?

A. Octile distance is the exact cost on an empty 8-connected grid; obstacles only lengthen routes, so it never overestimates. It is also consistent, which guarantees optimality with a closed set. (9.3)

Q. How much faster is A*?

A. On 40 random routes: 87 cells expanded against 974, 1.1 ms against 9 ms, with identical route lengths. (9.3)

Q. How does your RRT* work?

A. Two trees (from start and goal) with RRT* parent choice and rewiring within $\gamma\sqrt{(\log n / n)}$; cost savings are propagated to the subtree; informed sampling inside the ellipse once a route exists; shortcutting; Bresenham rasterisation; A* fallback if invalid. (9.4)

Q. What bug did you find in RRT*?

A. Tracing lines cell by cell produced staircases that cost up to 18 % more than the octile length. Bresenham fixed it ($1.013 \times$ optimum). Rewiring also didn't propagate cost savings to descendants; now a test checks $\text{cost}(\text{child}) = \text{cost}(\text{parent}) + \text{edge}$ for every node. (9.4, 9.5)

Q. Explain your ant colony.

A. 10 ants per round choose cells with probability $\propto \tau^\alpha \eta^\beta$ ($\alpha 1, \beta 4$), with $\eta = 1/(1 + \text{detour})$, where $\text{detour} = \text{step} + h(\text{next}) - h(\text{here})$. Pheromone evaporates 30 % per round, arriving ants deposit Q/L , the best path is reinforced (elitist), and it stops after 3 rounds without improvement. (9.5)

Q. Why is RRT* not better here?

A. It is built for continuous, high-dimensional spaces. On a small grid with narrow doors, random samples rarely hit the doorway, and grid search is exact and fast. (9.5)

Motion and robustness

Q. What happens when a new obstacle appears?

A. Every second the next 20 route cells are checked against the updated map. If one is now an obstacle, the robot replans immediately. If the sensors missed it, the bumper marks it as proven solid and the robot replans. (10.3)

Q. How do robots avoid each other?

A. Each round, robots state their next cell. No two in one cell and no swaps; a blocked robot waits, and after 3 s plans around teammates. With DWA, a local score (heading, clearance, progress, turn) picks a move around them instead. (10.1–10.2)

Q. How do you detect a dead end?

A. At most 4 different cells in the last 24 positions while driving, or blocked for 8 s. Recovery backs off at least 4 cells along the robot's own trail and puts the goal on a 60 s tabu list. (10.5)

Q. Why not forbid revisits?

A. Leaving a room often requires driving back through the same corridor. Avoid-revisits is a soft cost (0.3 per earlier visit, up to 5) that prefers fresh ground without making routes impossible. (10.6)

Q. How is time modelled?

A. Turn then drive: $t = |\Delta\theta|/\omega + d/v$, paid from a per-second time credit, so diagonals take $\sqrt{2}$ longer and sharp turns cost time. (2.5, 10.1)

Q. How is energy modelled?

A. Base power (computer 15 W, LiDAR 8 W, cameras 6 W), rolling resistance $m \cdot g \cdot C_{rr}/\eta = 20.4$ J/m, copper losses $k \cdot v$, and turning scrub. About 42 W at 0.5 m/s. (11.1)

Q. Can the mission run forever?

A. No. Every frontier is eventually searched, probed or given up; breakthrough attempts stop after 4 real obstacles; a robot whose battery can never reach the remaining tasks stays home; and missions stop at 900 s. (6.5, 10.3, 11.2)

Engineering

Q. How do you test a simulation?

A. With properties that have known answers: planners against Dijkstra's optimum; heuristic admissibility; spacing and ordering of patterns; robots actually driving through every reached waypoint; 100 % coverage for every navigation option; no flat batteries; agreement between robots; reproducibility. 92 tests in total. (13.4)

Q. How do you know your improvements are real?

A. Measure before and after against ground truth on fixed seeds, then confirm on held-out seeds not used for tuning, as with the confirmation rule. (14)

Q. How does the dashboard get its data?

A. A threaded Python HTTP server keeps one simulator per browser session. The page requests batches of steps and receives compact JSON (maps as strings and bits), then three.js draws it. (13.3)

Q. What would you do next?

A. Pose noise and localisation, message-based bidding over a lossy radio, sound as a second victim cue, footprint-aware continuous motion, and domain randomisation to test the detector on real images. (15.2)

17 Glossary and formula sheet

17.1 Glossary

Term	Meaning in this project
A*	best-first graph search ordered by $f = g + h$; optimal with an admissible, consistent h (octile distance here)
admissible heuristic	an estimate of remaining cost that never overestimates
ant colony optimisation	many simulated ants build paths guided by pheromone and a heuristic; pheromone rewards short paths and evaporates
auction (sequential)	repeatedly assign the (robot, task) pair with the highest utility, then update the others' utilities
belief / log-odds	$\ln(p/(1-p))$; evidence adds up; used for map cells and for victim candidates
boustrophedon	lawnmower coverage: parallel lanes in alternating directions
Bresenham line	the 8-connected cell chain of a straight segment; its length is the octile distance
bumper	contact sensor; driving into a cell the map wrongly called free marks it as proven solid
candidate / confirmed / rejected	states of a sighting in the belief filter
cell	a 0.5 m × 0.5 m square of the navigation grid (fine grid: 0.25 m)
CNN	convolutional neural network; here a small fully convolutional heatmap detector
corner cutting	a diagonal move past an obstacle's corner; never allowed
counted twice	a real person confirmed again more than 1.6 m from the first report
coverage (searched)	share of reachable floor seen by a camera
dead end / tabu	loop or blockage detection; back off and avoid that goal for 60 s
decoy / look-alike	an object that can fool a colour camera (jacket, bag, box, beam, barrel)
differential drive	two tracks or wheels; turns on the spot by driving them in opposite directions
dilated convolution	a convolution with gaps between its taps: a wider view for the same cost
Dijkstra	optimal shortest paths with non-negative costs; also the distance field used to pick goals
distance field	route distance from one cell to every cell (one Dijkstra run)
DWA	Dynamic Window Approach: score the admissible next moves by heading, clearance, progress and turn
evidence grid / occupancy grid	a map of per-cell belief of being occupied
free-space assumption	planning as if unknown cells were free (used for pattern waypoints)
frontier	a searched free cell next to unsearched space
ground truth	the real world state, known only to the simulator (for rendering and scoring)
Hungarian method	an optimal solver for one-to-one assignment (robots ↔ pattern regions)
informed sampling	RRT* samples only inside the ellipse where a shorter route could exist

inverse sensor model	turning a measurement into evidence about cells (beam passed → free, ended → occupied)
LiDAR	laser range finder: 240 distances per turn in a plane 40 cm above the floor
octile distance	$\max(dx, dy) + (\sqrt{2} - 1) \cdot \min(dx, dy)$: exact cost on an empty 8-connected grid
precision / recall	share of reports that were real / share of real victims that were reported
probe	drive into an unresolvable cell and let the bumper decide
ray casting / ray marching	following a line through the world until it hits a surface (LiDAR, camera)
RGB-D camera	colour camera with per-pixel depth
RRT*	sampling-based planner growing a tree of straight segments; asymptotically optimal with rewiring
seed	the starting number of a random generator; the same seed gives the same building and mission
SLAM	simultaneous localisation and mapping (not simulated here: positions are exact)
verify goal	a trip to look closely at an uncertain sighting
waypoint	a point on a coverage pattern, at most 1.5 m from the previous one

17.2 Formula sheet

What	Formula (defaults)
metres ↔ cells	cell = floor(m / 0.5); centre = (cell + 1/2) · 0.5 m
step cost	1 straight, $\sqrt{2} \approx 1.4142$ diagonal (+ 0.3 · min(visits, 5) with Avoid revisits)
move time	$t = \Delta\theta / 180^\circ/\text{s} + d / v$, $v = 0.5 \text{ m/s}$
octile heuristic	$h = \max(dx, dy) + 0.4142 \cdot \min(dx, dy)$
A* priority	$f(n) = g(n) + h(n)$
map evidence	$L = \Sigma (-0.45 \text{ per beam through, } +0.9 \text{ per beam ended})$, laser $ L \leq 2.5$, camera/bumper ± 4 ; FREE if $L \leq -0.8$, OBSTACLE if $L \geq 0.8$
victim evidence	$L = -0.6 + \Sigma w \cdot \ln(p/(1-p)) - \Sigma 0.9 w$, $w = 1 (\leq 2.5 \text{ m})$ or 0.55
confirm / reject	$L \geq 9$ and ≥ 6 frames and one look $\leq 2.5 \text{ m} / L \leq -1.5$
heat through rubble	$T = T_{\text{amb}} + (T_{\text{body}} - T_{\text{amb}}) \cdot e^{-d / 0.10 \text{ m}}$
auction utility	$u = \text{value} - 0.5 \cdot \text{distance(m)}$; value × d/4 m within 4 m of a won goal; taken within 2 m
verify value	30 × belief that the sighting is a person
lane spacing	$w = 2 \cdot R \cdot \sin(F/2) \cdot 0.75 \approx 5.3 \text{ m}$ (R 5 m, F 90°)
lanes	$K = \text{ceil}(D / w)$, lane k at $a_0 + (k + 1/2) \cdot D / K$
RRT* radius	$r = \gamma \cdot \sqrt{(\log n / n)}$, $\gamma = 2 \sqrt{(\text{free area} / \pi)}$, capped at $1.5 \times \text{step}$
ant choice	$P(j) \propto \tau_j \cdot \eta_j^4$, $\eta = 1 / (1 + c(i,j) + h(j) - h(i))$; $\tau \leftarrow 0.7 \tau + \Sigma 10 / L$
DWA score	$G = 0.45 \text{ heading} + 0.25 \text{ clearance} + 0.30 \text{ progress} - 0.10 \text{ turn}$

power	$P = 15 + 8 + 6 \text{ W}$; driving $25 \cdot 9.81 \cdot 0.05 / 0.6 + 12 \text{ v J/m} \approx 26.4 \text{ J/m}$; turning $\approx 38.5 \text{ J per } 90^\circ$
task is affordable	$1.3 (e d + P (d/v + 5 s)) + 1.3 (e d_{\text{home}} + P d_{\text{home}}/v) + 5 \% \text{ cap} \leq \text{left}$
turn home	$\text{left} \leq 1.3 (e d_{\text{home}} + P d_{\text{home}}/v) + 5 \% \text{ cap}$

WHERE TO GO FROM HERE

Run `python -m rescue serve`, pick the open test ground, and change one setting at a time while watching the Robots tab: every reason printed there corresponds to a section of this guide. Then read the code in the order of chapter 13's table. Each module's docstring summarises it in the same words as this guide.